Final Year Design Project

Hybrid Format Converter



Submitted By

Mubrra Amjad	**076922**
Aimen Zahid	**076953**
Umama Qamar	**077009**
Project ID	**BSIT22-G28**

Project Advisor:

Prof.Usman Ahmad

Faculty of Computing & Information Technology

Govt. Graduate Islamia College, Gujranwala.

(2022-2026)

Hybrid Format Converter

A Project represented to

University of the Punjab, Lahore.

In Partial fulfilment of

the requirement for the degree of

Bachelor of Science in Information Technology,

Session (2022-2026).

By

Aimen Zahid	**2022-ICG-164**
Mubrra Amjad	**2022-ICG-177**
Umama Qamar	**2022-ICG-187**

FACULTY OF COMPUTING &

INFORMATION TECHNOLOGY,

UNIVERSITY OF THE PUNJAB, LAHORE.

DECLARATION

We hereby declare that this software, neither the whole nor a part, was copied from any source. It is further declared that we have developed this software and accompanying report based on our efforts. If any part of this project is proven to be copied from any source or found to be a reproduction of some other, we will stand by the consequences. No portion of the work presented has been submitted for any application for any other degree or qualification of this or any other university or institute of learning.

Mubrra Amjad [076922] Aimen Zahid [076953] Umama Qamar [077009]

Signature: _____ Signature: _____ Signature: _____

CERTIFICATE OF APPROVAL

It is to certify that the final year project (FYP) of BSIT, session (2022-2026), the project Hybrid Format Converter Application was developed by Aimen Zahid (2022-ICG-164), Mubrra Amjad (2022-ICG-177), and Umama Qamar (2022-ICG-187) under the supervision of Prof. Muhammad Usman in my opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Information Technology.

FYP Advisor: Prof. Muhammad Usman

Advisor Signature: _____

Signatures (Faculty Advisory Committee (FAC))

	FAC1	**FAC2**
Name	**Prof. Usman Ahmad**	**Prof. Muhammad Asif**
Signature		

Head of FYP Coordination Office: Prof. Muhammad Usman

Signature: _____

Head Of Department, Information Technology: Prof. Usman Ahmad

Signature: _____ Dated: _____

Executive Summary

Hybrid Format Converter is an android mobile application built to make file conversions along with other file operations easier for academic or other purposes. The app provides conversion formats for images, documents, audios and videos online and offline. The system aims to fulfill the need of file conversion whenever there is no internet. It enables the user to access their last successfully processed files. It also provides a feedback mechanism where user can give feedback to the system to improve usability. It provides text extraction feature through OCR for online as well as offline text extraction. It also provides additional features like compression and PDF tools to improve functionality. The app combines multiple functionalities such as image/audio/video/documents conversion, compression, Text extraction and PDF tools in one place. There is no need of separate application for these. Many features work without internet which makes it usable in case of unstable or no internet connection.

The system is designed using technologies like Firebase Firestore for feedback storage and room database which is a relational for data storage. The system also uses external APIs for online text extraction and conversion. Hybrid Format Converter combines multiple essential tools into a single platform that eliminates the need for many separate applications. App's usefulness increases as it provides offline functionalities in areas with no internet connection. The app solve everyday problems like converting & compressing files, splitting/merging PDF and extracting text.

Acknowledgment

We express our gratitude to our Almighty Allah, and after that, we are thankful to our families, who allowed us to get higher education, and we became capable of creating something productive. We are also thankful to our project advisor and department faculty for their support.

Mubrra Amjad [076922] Aimen Zahid [076953] Umama Qamar [077009]

Signature: _____ Signature: _____ Signature: _____

Abbreviation

API	Application Programming Interface: Allows software applications to communicate with each other.
ER	Entity Relationship: Visual representation of entities and their relationship
UT	Unit Testing: a technique in which components are tested independently
FT	Functional Testing: Verifies that the system works according to the specified requirements by checking the feature.
IT	Integration Testing: multiple components are combined and tested together
PT	Performance Testing: It evaluates how a system perform under specific conditions
NO SQL	Not only SQL: database which stores data in non-tabular formats.
MVVM	Model View ViewModel: an architecture pattern that separates an app Model (data), View (UI) and ViewModel (Logic)
UI	User interface: a part of application that users interact with
XML	Extensible Markup Language: used to transport data in a structured and readable format.
OCR	Optical Character Recognition: a technology that convert images of text into editable text.
RAM	Random Access Memory, a type of memory used to temporarily store data.
NFR	Non- Functional Requirements. Defines how a system should perform.
DB	Database: used to store data in an organized manner
UC	Use Case, describes how a user interacts with a system to achieve a specific goal.
IT	Information Technology
PU	Punjab University
SDS	Software Design Specification, a document that describes how a software will be built including architecture and components.
RTM	Requirement Traceability Matrix, a matrix that maps and tracks requirements.

Table of Contents

Executive Summary 5

1. Introduction 18

1.1 Problem Statement 18

1.2 Problem Solution 18

1.3 Objectives of the Proposed System 19

1.4 Scope 19

1.5 System Components 19

1.5.1 Feature 1: Conversion (End User Mobile Application) 20

1.5.2 Feature 2: Compression (End User Application) 20

1.5.3 Feature 3: Text Extraction (End User Application) 20

1.5.4 Feature 4: Pdf Tools (End User Application) 20

1.5.5 Feature 5: History (End User Application) 21

1.5.6 Feature 6: Feedback (End User Mobile Application) 21

1.6 Related System Analysis/Literature Review 21

1.7 Vision Statement 22

1.8 System Limitations and Constraints 22

1.9 Tools and Technologies 23

1.10 Work division 24

1.11 Project Deliverables 24

1.12 Project Planning 24

1.13 Summary 25

2. Analysis 27

2.1 User Classes and Characteristics 27

2.2 Requirement Identifying Technique 27

2.2.1 Surveys 27

2.2.2 Observation 27

2.2.3 Use-Case Modeling 28

2.3 Functional Requirements 28

2.3.1 Functional Requirement 1 28

2.3.2 Functional Requirement 2 29

2.3.3 Functional Requirement 3 29
2.3.4 Functional Requirement 4 30
2.3.5 Functional Requirement 5 31
2.3.6 Functional Requirement 6 32
2.3.7 Functional Requirement 7 32
2.3.8 Functional Requirement 8 33
2.3.9 Functional Requirement 9 34
2.3.10 Functional Requirement 10 34
2.3.11 Functional Requirement 11 35
2.4 Non- Functional Requirements 36
2.5 Use Case Analysis 37
2.5.1 Use Case 1 37
2.5.2 Use Case 2 38
2.5.3 Use Case 3 39
2.5.4 Use Case 4 40
2.5.5 Use Case 5 41
2.5.6 Use Case 6 41
2.5.7 Use Case 7 42
2.5.8 Use Case 8 43
2.5.9 Use Case 9 44
2.5.10 Use Case 10 45
2.5.11 Use Case 11 46
2.5.12 Use Case Diagram 48
2.6 External Interfaces 48
2.6.1 User Interface Requirements 48
2.6.2 Software Interfaces 49
2.6.3 Hardware Interfaces 49
2.6.4 Communication Interfaces 50
2.7 Summary 50
3. System Design 52
3.1 Design Considerations 52

3.1.1 Assumption and dependencies 52
3.1.2 Limitations: 52
3.1.3 Risks 53
3.2 Design Models 53
3.2.1 Class Diagram 53
3.2.2 Sequence Diagram 55
3.2.3 State Transition Diagram 61
3.3 Architectural Design 67
3.3.1 Component Diagram 67
3.3.2 Architectural Diagram 68
3.4 Data Design 68
3.4.1 Data Dictionary 71
3.5 User Interface 73
3.5.1 Screen Images 75
3.5.2 Screen Objects and Actions 75
3.6 Behavioral Model 76
3.7 Design Decisions 78
3.8 Summary 78
4. Implementation 80
4.1 Algorithms 80
4.1.1 Algorithm 1 File Permission Request 80
4.1.2 Algorithm 2 File Conversion 81
4.1.3 Algorithm 3 File Compression 82
4.1.4 Algorithm 4 Text Extraction 83
4.1.5 Algorithm 5 Pdf Merge 83
4.1.6 Algorithm 6 Pdf split 84
4.1.7 Algorithm 7 Download File 84
4.1.8 Algorithm 8 Share File 85
4.1.9 Algorithm 9 Delete File 85
4.1.10 Algorithm 10 Rename File 86
4.1.11 Algorithm 11 Load History 87

4.1.12 Algorithm 12 Feedback and Ratings Submission 87
4.2 External APIs/SDKs 88
4.3 Code Repository 89
4.4 Summary 89
5. Evaluation and Testing 91
5.1 Unit Testing (UT) 91
5.1.1 Test Case 1 91
5.1.2 Test Case 2 92
5.1.3 Test Case 3 93
5.1.4 Test Case 4 94
5.1.5 Test Case 5 95
5.1.6 Test Case 6 96
5.1.7 Test Case 7 96
5.1.8 Test Case 8 97
5.1.9 Test Case 9 98
5.2 Functional Testing (FT) 99
5.2.1 Test Case 1 99
5.2.2 Test Case 2 100
5.2.3 Test Case 3 101
5.2.4 Test Case 4 102
5.2.5 Test Case 5 103
5.2.6 Test Case 6 104
5.2.7 Test Case 7 105
5.3 Integration Testing 106
5.3.1 Test Case 1 106
5.3.2 Test Case 2 107
5.3.3 Test Case 3 108
5.3.4 Test Case 4 109
5.3.5 Test Case 5 110
5.4 Performance testing 111
5.4.1 Test Case 1 114

5.4.2 Test Case 2 115
5.4.3 Test Case 3 116
5.4.4 Test Case 4 117
5.5 Summary 118
6. System Conversion 120
6.1 Conversion method 120
6.1.1 Phased Conversion 120
6.1.2 Phases Distribution 121
6.2 Deployment 121
6.2.1 Deployment Steps 122
6.2.2 Data Conversion 123
6.2.3 Training 124
6.3 Post Development Testing 125
6.3.1 Post Deployment Testing Steps 125
6.4 Challenges 126
6.5 Summary 127
7. Conclusion 129
7.1 Evaluation 129
7.2 Traceability Matrix 130
7.3 Conclusion 131
7.4 Future Works 132
References 133
Appendix A: Use Case Description 134
Appendix B: Coding Standards 144
1. Variable Naming Conventions 144
2. Visual Scope Indicators 144
3. Read-Only and Immutable Fields 144
4. Function Naming and Behavior 144
5. Commenting Style 145
6. Indentation and Code Structure 145
7. File and Folder Naming 145

8. Class Structure and Separation of Concerns 146
9. Access Modifiers and Scoping 146
10. Cohesion and coupling 146
11. DRY Principle 146
12. Single Responsibility Principle 147
13. Extensibility and Maintainability 147
14. Exception Handling and Logging 147
Appendix C: Application Prototype 148
Application Overview 148
Screenshots of key Interfaces 148
Appendix D: Application's provided formats 150
Conversion 150
Compression 151
Text Extraction 152
PDF Tools 152

List of Figures

Figure 1-1 Gant Chart 24
Figure 2-1 Use Case Diagram 48
Figure 3-1 Class Diagram 54
Figure 3-2 Sequence Diagram for Conversion Module 56
Figure 3-3 Sequence Diagram for Compression Module 57
Figure 3-4 Sequence Diagram for Text Extraction Module 58
Figure 3-5 Sequence Diagram for Pdf Tools Module 59
Figure 3-6 Sequence Diagram for History Module 60
Figure 3-7 Sequence Diagram for Feedback Module 61
Figure 3-8 State Diagram for Conversion Module 62
Figure 3-9 State Diagram for Compression Module 63
Figure 3-10 State Diagram for Text Extraction Module 64
Figure 3-11 State Diagram for Pdf Tools Module 65
Figure 3-12 State Diagram for Feedback Module 66
Figure 3-13 State Diagram for History Module 66
Figure 3-14 Component Diagram 67
Figure 3-15 Architectural Diagram 68
Figure 3-16 ER Diagram 70
Figure 3-17 Screen Images 75
Figure 3-18 Screen Objects 75
Figure 3-19 Overall State Diagram 77
Figure 5-1 Testing Sample 1 113
Figure 5-2 Testing Sample 2 113
Figure 5-3 Testing Sample 3 114

List of Tables

Table 1 Related System 22
Table 2 Tools and Technologies 23
Table 3User classes and characterization 27
Table 4 Functional Requirement 1 28
Table 5 Functional Requirement 2 29
Table 6 Functional Requirement 3 29
Table 7 Functional Requirement 4 30
Table 8 Functional Requirement 5 31
Table 9 Functional Requirement 6 32
Table 10 Functional Requirement 7 32
Table 11 Functional Requirement 8 33
Table 12 Functional Requirement 9 34
Table 13 Functional Requirement 10 34
Table 14 Functional Requirement 11 35
Table 15 Non-Functional Requirement 36
Table 16 Use Case 1 37
Table 17 Use Case 2 38
Table 18 Use Case 3 39
Table 19 Use Case 4 40
Table 20 Use Case 5 41
Table 21 Use Case 6 41
Table 22 Use Case 7 42
Table 23 Use Case 8 43
Table 24 Use Case 9 44
Table 25 Use Case 10 45
Table 26 Use Case 11 46
Table 27 Data Organization 69
Table 28 Data Dictionary 71
Table 29 Structured Approach 72
Table 30 Object Oriented Approach 72
Table 31 Algorithm 1 80
Table 32 Algorithm 2 81
Table 33 Algorithm 3 82
Table 34 Algorithm 4 83
Table 35 Algorithm 5 83
Table 36 Algorithm 6 84
Table 37 Algorithm 7 84
Table 38 Algorithm 8 85
Table 39 Algorithm 9 85
Table 40 Algorithm 10 86
Table 41 Algorithm 11 87
Table 42 Algorithm 12 87
Table 43 External APIs/SDKs 88
Table 44 Unit Test Case 1 91

Table 45 Unit Test Case 2 92
Table 46 Unit Test Case 3 93
Table 47 Unit Test Case 4 94
Table 48 Unit Test Case 5 95
Table 49 Unit Test Case 6 96
Table 50 Unit Test Case 7 96
Table 51 Unit Test Case 8 97
Table 52 Unit Test Case 9 98
Table 53 Functional Test Case 1 99
Table 54 Functional Test Case 2 100
Table 55 Functional Test Case 3 101
Table 56 Functional Test Case 4 102
Table 57 Functional Test Case 5 103
Table 58 Functional Test Case 6 104
Table 59 Functional Test Case 7 105
Table 60 Integration Test Case 1 106
Table 61 Integration Test Case 2 107
Table 62 Integration Test Case 3 108
Table 63 Integration Test Case 4 109
Table 64 Integration Test Case 5 110
Table 65 Performance Testing Metrics 112
Table 66 Performance Test Case 1 114
Table 67 Performance Test Case 2 115
Table 68 Performance Test Case 3 116
Table 69 Performance Test Case 4 117
Table 70 Phase Distribution 121
Table 71 Deployment Steps 122
Table 72 Data Conversion 123
Table 73 Training Instruction 1 124
Table 74 Training Instruction 2 124
Table 75 Post Deployment Testing Steps 125
Table 76 Challenges and Solutions 126
Table 77 Progress Evaluation 129
Table 78 Traceability Matrix 130
Table 79 Objectives and System Functionalities 131

Chapter 1

Introduction

1. Introduction

This chapter highlights the growing demand for efficient file conversion without internet dependency. Many existing apps or webs rely on online processing, making it difficult for users to access it at anytime and anywhere. Our Hybrid Format Converter aims to address these challenges by providing offline conversion, along with additional features such as compression and text extraction. The app is developed using various tools and technologies with its constraints discussed in following section.

1.1 Problem Statement

People find it complicated to visit multiple websites and applications for conversion of different formats, compression and text extraction. It becomes even more challenging when internet access is unavailable. The lack of offline conversion options forces users to rely heavily on online tools, which may not always be accessible. No availability of offline conversions and switching between multiple webs and apps for multiple tasks often lead to inconvenience and might leave the user in frustration. Most of the existing applications have complex UIs that are not user friendly with beginners. Some of the applications are ad heavy that disrupt the user experience. Moreover, some applications have restricted the advanced functionalities in premium versions and also demand continuous feedback pop ups. According to survey we have done, 67% people feel that an offline converter is necessary. Furthermore, 6 out of 10 people convert their files frequently. There is a pressing need for a hybrid converter app with features such as basic offline conversion and compression in simple user-friendly interference.

1.2 Problem Solution

We are developing an Android application named “Hybrid Format Converter”. This application provides various formats for conversion, compression, along with text extraction. It is designed to simplify the user experience by integrating various functionalities into a single platform. Most of the conversions are performed offline, reducing internet dependency. However, DOCX to PDF and PDF to DOCX conversions require an internet connection, while the remaining supported conversions are performed offline. Text extraction feature works online. Compression feature works entirely offline. It also enables feedback mechanism where user can share their experience and suggestions. Users can also request new functionalities through feedback. The system also maintains history that display last 5 download files. The application also offers pdf tools such as pdf merging and pdf splitting and the conversion between image and pdf. Overall, the Hybrid Format Converter aims to provide all-in-one solution for file conversion, compression and extraction.

1.3 Objectives of the Proposed System

1. To design and develop a hybrid Android application for files conversions, compression and text extraction.
2. To provide conversion formats for images, documents, audios and videos.
3. To provide both offline and online conversion capabilities according to the requirements of different file formats.
4. To implement history feature where end-users can access their last 5 downloaded files.
5. To ensure offline conversion so user can perform conversion with no internet access.
6. To provide text extraction feature using Optical Character Recognition (OCR) to improve usability.
7. To implement compression feature to reduce the size of large files.
8. To provide pdf tools such as pdf merging and pdf splitting.
9. To provide a simple and user-friendly user interference.
10. To include a feedback mechanism for user to share suggestion or request new features.

1.4 Scope

The proposed project aims to develop a hybrid mobile application especially for students, teachers and office workers. The system will include modules for file conversion, compression, text extraction and pdf tools such as pdf merge and pdf split. The application will be developed using Java/Android Studio and different libraries. The system will not include every existing format for file conversion and compression and offline conversions will be limited, in its first phase. The initial phase will focus on maximum offline conversions and text extraction.

Target Stakeholders include

- Students
- Professors/Teachers
- Office workers
- Developers

Limitations:

- It will not include the conversion of all existing formats.
- All the provided conversions will not be offline.
- Compression feature will not support all the formats.

1.5 System Components

The application consists of a single main module, End User Mobile Application. The system is primarily designed to operate offline, allowing user to perform most functionalities without internet access. However, some conversions and text extraction require internet access. Data

storage is handled locally on user's device to ensure offline functionality. However, feedback is not stored locally, instead it is submitted online and stored in Firebase Firestore.

1.5.1 Feature 1: Conversion (End User Mobile Application)

The list of features Conversion will have and how it will be used is as follows.

1. Home screen will display different types of conversion.
2. User can choose a file type such as images audios, videos and documents.
3. A list of all available formats for conversion will be displayed.
4. User will select the desired conversion format and upload the file for conversion.
5. The conversion process will be performed using libraries.
6. The converted file will be displayed to the user for download.
7. The downloaded file will automatically be stored in history.

1.5.2 Feature 2: Compression (End User Application)

The list of features Compression will have and how it will be used is as follows:

1. From home screen, user will select Compression feature.
2. A list of all available formats for compression will be displayed.
3. User will select the desired compression format and upload the file for compression.
4. Compression will be performed without internet connectivity.
5. The compressed file will be displayed to the user for download.
6. The downloaded file will automatically be stored in history.

1.5.3 Feature 3: Text Extraction (End User Application)

The list of features Text Extraction will have and how it will be used is as follows:

1. From home screen, User will select text extraction feature.
2. A list of all available formats for text extraction will be displayed.
3. User will select the desired text extraction format and upload the file for text extraction.
4. Text extraction will be performed using APIs to extract the text from supported files.
5. The extracted text will be displayed to the user, with options to copy the text or save it as a .txt file.
6. The downloaded file will automatically be stored in history.

1.5.4 Feature 4: Pdf Tools (End User Application)

The list of features Pdf Tools will have and how it will be used is as follows:

1. From home screen, User will select pdf tools feature.
2. Pdf Tools such as pdf merge and pdf split will be displayed.
3. User will upload the file that he wants to merge or split.
4. The Pdf merge/split file will be displayed to user for download.
5. The downloaded file will automatically be stored in history.

1.5.5 Feature 5: History (End User Application)

The list of features History will have and how it will be used is as follows:

1. The screen will have three navigation options home, history and settings.
2. In history last 5 downloaded files will be displayed.
3. History will be stored in Room database.
4. User can download any file from last 5 downloaded files.
5. User can also rename or share any file from history.
6. User can also delete file from history.

1.5.6 Feature 6: Feedback (End User Mobile Application)

The list of features Feedback will have and how it will be used is as follows:

1. The screen will have three navigation options home, history and settings.
2. From settings, User can access the feedback feature.
3. User can either submit the written feedback or 5-star ratings.
4. For written feedback, user needs to fill the input text box.
5. All the feedback is collected anonymously and will be stored online in Firestore Firebase

1.6 Related System Analysis/Literature Review

Our Project focuses on offline functionality within Hybrid Format Converter. No other popular application or website that offers this level of offline availability for conversion functionalities. All features are free, Our UI is easy to use, user can navigate through application with ease. Mostly Converter applications like All File Converter, All In One File Converter, File Converter, Fexo File Converter do not provide offline conversions, compression and text extraction.

Table 1 Related System

Application Name	Weakness	Proposed Solution
All File Converter	It requires stable internet connection for conversion and also has a complex UI.	Hybrid Format Converter provides offline functionality for conversion of images, documents, audios and videos.
All In One File Converter	It is ad-heavy and needs continuous internet connection	Provides most of the conversion offline.
File Converter	It works completely online and does not provide any compression feature.	Hybrid Format Converter works offline for many conversions and also provide compression feature.
Fexo File Converter	Conversion formats are very limited and does not provide compression feature.	Hybrid Format Converter provides many conversion formats and also offers compression feature.

1.7 Vision Statement

For teachers, Students, office workers and other mobile users **who** want to take advantage of conversion, compression, text extraction without internet access **the** Hybrid Format Converter **is** an Android application **that** provide facility of converting different formats without internet access **unlike** other format converter applications that require constant internet even for basic conversion, compression and text extraction **our product** is free, easy to use and provide most of the conversions in offline mode.

1.8 System Limitations and Constraints

The system will have following limitations:

1. In feedback module, Feedback is stored on cloud storage. We are using Firebase Firestore to save the feedback of users to improve the functionality or usability.

2. We are using external APIs for text extraction and to perform conversion on advance formats. So, Internet is required for conversion of advance formats.

The system will have following constraints:

1. It is an android application so it will not have iOS and desktop version.
2. Compression feature will not support all formats.
3. Text Extraction will not be offline.
4. All provided conversions will not be offline.
5. All existing formats for conversion will not be included in this application.

1.9 Tools and Technologies

Our system will use following tools and technologies for complete implementation of goals/objectives:

Table 2 Tools and Technologies

	Tools	Version	Rationale
Tools, Libraries and Technologies	Android Studio	Quail 2026.1.1	It is an official IDE for android app development.
	Technology	**Version**	**Rationale**
	Java	11	Most of the android apps are developed in Java.
	XML	Latest	For Layouts
	OCR	Latest	To extract online.
	Libraries	**Version**	**Rationale**
	FFmeg /FFmpegKit	6.1.1	Used for audios, videos, images conversions.
	PdfBox	2.0.27.0	Used for pdf processing and manipulation.
	OkHttp	4.12.0	Used for making HTTP requests.
	RoomDatabase	2.6.1	Used for local database storage.

	Apache POI	v5.2.5	To extract text from DOCX/PPT without internet connection.

1.10 Work division

Sr. No	Roll Numbers	Names	Role Assignment and Work Division
1.	076922	Mubrra Amjad	Frontend +Backend+ Documentation
2.	076953	Aimen Zahid	Frontend +Backend+ Documentation
3.	077009	Umama Qamar	Frontend +Backend+ Documentation

1.11 Project Deliverables

Our system will have following deliverables by the end of 7th semester:

1. Project proposal
2. Software Design Specification (SDS)
3. Requirement Traceability Matrix (RTM)
4. Prototype

Our system will have following deliverables by the end of 8th semester:

1. Coding/Implementation
2. Testing
3. Deployment
4. Final Documentation

1.12 Project Planning

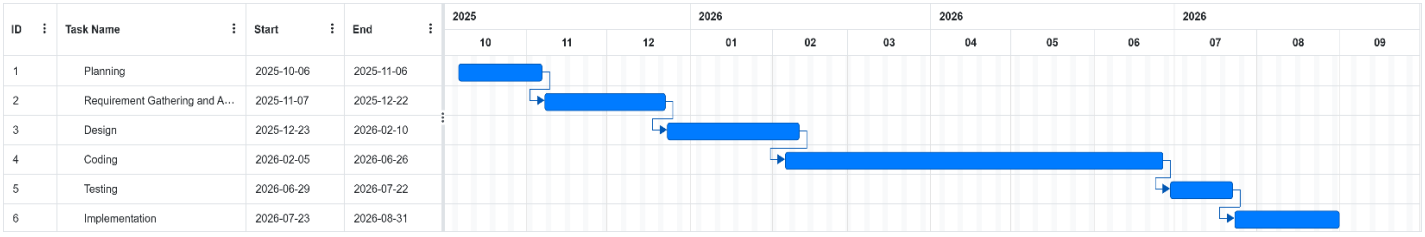


Figure 1-1 Gant Chart

1.13 Summary

To address the challenges faced by users in file conversion, compression, and text extraction, our project Hybrid Format Converter provides an all-in-one solution. Our project will deliver its modules and fulfill its goals and objectives within the planned timeframe while maintaining the scope of project to deliver a quality product.

Chapter 2

Requirement Analysis

2. Analysis

This chapter focuses on requirement identification, elicitation and analysis. The system consists of a single user class i.e. End Users. For requirements identification and elicitation research-based method were used, surveys were conducted to understand user's need. Also, identified requirement are categorized into functional and non-functional requirements, along with their use cases and description.

2.1 User Classes and Characteristics

Table 3User classes and characterization

User Class	Characteristics
End Users	The end users would be student, teacher or any general user who needs to convert their file. They use the system to convert, compress and extract the text into different formats for work-related tasks.

2.2 Requirement Identifying Technique

There are fundamental approaches used to identify requirements. As elicitation of requirements requires meeting with stakeholders so following are the approaches used:

2.2.1 Surveys

The survey with no pre-defined criteria was conducted with stakeholders like students, teachers and any general user to gather and understands the requirement of users. It helps us to understand what user wants from application.

2.2.2 Observation

It is a technique in which developers observe and interact with existing similar applications. It helps to understand user behavior, user requirement and what common problems user have with existing systems.

2.2.3 Use-Case Modeling

The development of use cases and scenarios help describe how the system will be used in different situations. This technique helps in understanding the interactions between users and the system, making it easier to identify and document functional requirements. Use cases are a way of describing interactions between users and a system.

2.3 Functional Requirements

This section describes the functional requirements of the system expressed in the natural language style. This section is organized by features as a system feature name and specific functional requirements associated with this feature. So, for each module and it's feature the functional requirements are itemized below:

2.3.1 Functional Requirement 1

Table 4 Functional Requirement 1

Identifier	FR-1
Title	Upload File
Requirement	The system shall allow the end users to upload the file to perform any operation.
Source	User Requirement/Observation
Rationale	Required as input for system functionalities.
Business Rule	Only supported file format should be accepted.
Dependencies	File storage system
Priority	High

2.3.2 Functional Requirement 2

Table 5 Functional Requirement 2

Identifier	FR-2
Title	File Conversion
Requirement	The system shall allow the end users to convert an uploaded file into a selected format.
Source	Business Requirement
Rationale	Enable the core functionality of Hybrid Format Converter, allowing the user to transform file between different formats.
Business Rule	Conversion must be performed on supported format.
Dependencies	FR-1, libraries for conversions
Priority	High

2.3.3 Functional Requirement 3

Table 6 Functional Requirement 3

Identifier	FR-3
Title	File Compression

Requirement	The system shall allow the end users to reduce the size of their files by compression feature.
Source	Business Requirement
Rationale	To optimize the storage.
Business Rule	Compression must not reduce the file quality.
Dependencies	FR-1, Libraries for compression
Priority	High

2.3.4 Functional Requirement 4

Table 7 Functional Requirement 4

Identifier	FR-4
Title	Text Extraction
Requirement	The system shall allow the end users to extract the text from uploaded files.
Source	Business Requirement
Rationale	To enable the user to extract the text from different file formats.
Business Rule	The system must provide text extraction through online OCR services, and an active internet connection must be available to perform text extraction.

Dependencies	FR-1, Internet connection(online)
Priority	High

2.3.5 Functional Requirement 5

Table 8 Functional Requirement 5

Identifier	FR-5
Title	Pdf Tools
Requirement	The system shall allow the end users to perform Pdf related functions such as Pdf merge and Pdf split.
Source	Business Requirement
Rationale	To manipulate the Pdf files.
Business Rule	The system must allow the merging of multiple Pdf files and splitting of Pdf.
Dependencies	FR-1, offline Libraries
Priority	High

2.3.6 Functional Requirement 6

Table 9 Functional Requirement 6

Identifier	FR-6
Title	Submit Feedback
Requirement	The system shall allow the end users to submit feedback to the system in terms of performance.
Source	Business Requirement, Observation
Rationale	To improve the overall functionality of a system.
Business Rule	Feedback must include text box or ratings.
Dependencies	Firebase Firestore, Internet Connection
Priority	Medium

2.3.7 Functional Requirement 7

Table 10 Functional Requirement 7

Identifier	FR-7
Title	History
Requirement	The system shall allow the end users to access the last downloaded files.

Source	Business Requirement
Rationale	To allow the user to track their successfully processed files.
Business Rule	Must include the file name.
Dependencies	FR-2, FR-3, FR-4, FR-5, Database for storage.
Priority	High

2.3.8 Functional Requirement 8

Table 11 Functional Requirement 8

Identifier	FR-8
Title	Download the processed file
Requirement	The system shall allow the end users to download the processed files after successful conversion, compression, text extraction and Pdf tools.
Source	User Requirement/Observation
Rationale	To enable users to save and use the final output file on their local device for further use.
Business Rule	The file must be available for download after successful processing.
Dependencies	FR-1, FR-2, FR-3, FR-4, FR-5

Priority	High

2.3.9 Functional Requirement 9

Table 12 Functional Requirement 9

Identifier	FR-9
Title	Rename file
Requirement	The system shall allow the end users to rename the file stored in history.
Source	User Requirement, Observation
Rationale	To enable the user to manage their files by meaningful names.
Business Rule	The new file name must not be empty.
Dependencies	Fr-7, storage system
Priority	Medium

2.3.10 Functional Requirement 10

Table 13 Functional Requirement 10

Identifier	FR-10
Title	Delete File

Requirement	The system shall allow the end users to delete the records of their successfully processed files from the history.
Source	User Requirement/Observation
Rationale	To allow the user to manage the storage space.
Business Rule	Deleted records must be permanently removed from the database.
Dependencies	FR-7
Priority	Medium

2.3.11 Functional Requirement 11

Table 14 Functional Requirement 11

Identifier	FR-11
Title	Share File
Requirement	The system shall allow the end users to share the processed files.
Source	User Requirement
Rationale	To unable the user to conveniently distribute processed files to others.
Business Rule	Only successfully processed file may be shared.

Dependencies	FR-1, FR-2, FR-3, FR-4, FR-5
Priority	Medium

2.4 Non- Functional Requirements

These are constraints on the services or functions offered by the system. They include timing constraints, constraints on the development process, and constraints imposed by standards. Non-functional requirements often apply to the system as a whole rather than individual system features or services. Following are non-functional requirements:

Table 15 Non-Functional Requirement

Identifier	Category	Requirement Description	Priority
NFR-1	Usability	The application shall be easy to use for general users.	Medium
NFR-2	Usability	The system shall show the progress indicator while operation is being performed.	Medium
NFR-3	Performance	Small files shall be converted or compressed within the 5-10 seconds depending on the device performance.	High
NFR-4	Security	User's files will be accessed only when the user grant permission to access them.	High
NFR-5	Reliability	The system shall perform smoothly without internet connection for offline operation, ensuring availability.	High

NFR-6	Availability	The system shall be available to users whenever needed, especially for online features requiring APIs.	High

2.5 Use Case Analysis

The systems involve interaction of some kind. This can be user interaction, which involves user inputs and outputs, interaction between the software being developed and other systems in its environment, or interaction between the components of a software system. User interaction modeling is important as it helps to identify user requirements.

2.5.1 Use Case 1

Table 16 Use Case 1

Use Case ID	UC-1
Use Case Name	Perform File Conversion
Actors	End User
Description	This use case allows the user to convert the uploaded file in different formats.
Trigger	User clicks “Convert”.
Pre-Conditions	The user must upload the file.
Post Condition	The converted file will be generated and available for user to download.
Normal Flow	1. User opens the app. 2. User selects the conversion. 3. User selects the I/O format. 4. User will upload the file. 5. User selects “Convert”. 6. File is converted and available to download.

Alternative Flows	If conversion failed, user will be shown an error message.
Business Rules	Only supported file will be converted.
Assumptions	• Device has enough storage. • Device permission for file access is granted.

2.5.2 Use Case 2

Table 17 Use Case 2

Use Case ID	UC-2
Use Case Name	Perform File Compression
Actors	End User
Description	This use case allows the user to compress the uploaded file in different formats.
Trigger	User clicks “Compress”.
Pre-Conditions	The user must upload the file.
Post Condition	The compressed file will be generated and available for user to download.
Normal Flow	1. User opens the app. 2. User selects the compression. 3. User selects the format. 4. User will upload the file. 5. User selects “Compress”. 6. File is compressed and available to download.
Alternative Flows	If compression failed, user will be shown an error message.
Business Rules	Only supported file will be compressed.

Assumptions	• Device has enough storage. • Device permission for file access is granted.

2.5.3 Use Case 3

Table 18 Use Case 3

Use Case ID	UC-3
Use Case Name	Text Extraction
Actors	End User
Description	This use case allows the users to extract the text from supported files.
Trigger	User clicks “Text Extract”.
Pre-Conditions	The user must upload a supported file for text extraction.
Post Condition	The extracted text will be displayed to the user, and the user can either copy the extracted text or save it as a .txt file.
Normal Flow	1. User opens the app. 2. User selects the Text Extraction. 3. User selects the format. 4. User will upload the file. 5. User selects “Text Extract”. 6. Text is extracted and saved in text file which is available to download.
Alternative Flows	If Extraction failed, user will be shown an error message.
Business Rules	Only supported files will be extracted.
Assumptions	• Device permission for file access is granted. • Internet is available for online Text Extraction if required.

2.5.4 Use Case 4

Table 19 Use Case 4

Use Case ID	UC-4
Use Case Name	Pdf Tools
Actors	End User
Description	This use case allows the end users to perform Pdf operations.
Trigger	User selects the "Merge" for merging the Pdf and "Split" for splitting the Pdf.
Pre-Conditions	User must upload the multiple Pdf files for merging and single Pdf file to split.
Post Condition	Single pdf will be generated in the result of pdf merge and multiple pdfs will be generated in the outcome of pdf split.
Normal Flow	1. User opens the app. 2. User selects the Pdf Tools. 3. User selects desired tool. 4. User will upload single or multiple Pdfs for different operation. 5. User selects the "Merge" for merging the Pdf and "Split" for splitting the Pdf. 6. Merged pdf or split pdfs will be available for user to download.
Alternative Flows	If operation failed, user will be shown an error message.
Business Rules	Only Pdf files are allowed.
Assumptions	• Device permission for file access is granted. • User must have more than one pdf for merging.

2.5.5 Use Case 5

Table 20 Use Case 5

Use Case ID	UC-5
Use Case Name	Submit Feedback
Actors	End User
Description	This use case allows the user to give feedback or ratings to the system.
Trigger	User selects “Submit Feedback”.
Pre-Conditions	Application must be installed and running.
Post Condition	User’s feedback will be stored on Firebase Firestore.
Normal Flow	1. User opens the application. 2. User navigated to the settings screen. 3. User selects written feedback or ratings. 4. For written feedback, user will be shown a text box to fill while for ratings, user will be shown 5 stars to fill. 5. User fills the text box or ratings. 6. User selects “Submit Feedback”
Alternative Flows	If internet is not available user will be shown an error message.
Business Rules	Text box for written feedback and 5 stars cannot be empty.
Assumptions	Internet is available to store the feedback in Firebase Firestore.

2.5.6 Use Case 6

Table 21 Use Case 6

Use Case ID	UC-6
Use Case Name	View History

Actors	End User
Description	This use case allows the user to access previously processed files including conversion, compression, text extraction and pdf tools.
Trigger	User selects the "History" from navigation menu.
Pre-Conditions	The user must have previously performed at-least one file operation.
Post Condition	The user can view and access their file processing history records.
Normal Flow	1. User opens the application. 2. User navigated to the history screen. 3. System retrieves the previously downloaded file records. 4. User can download, rename, delete and share history records.
Alternative Flows	If no previous record exists, system displays an empty history message.
Business Rules	History must store file operation records. User may mange history records through available action.
Assumptions	• The system has successfully stored previous operations. • User data is maintained.

2.5.7 Use Case 7

Table 22 Use Case 7

Use Case ID	UC-7
Use Case Name	Select file
Actors	End User
Description	This use case allows the user to upload the file from device to perform desired operation.

Trigger	User clicks “Select File”.
Pre-Conditions	File must be available for user to upload.
Post Condition	Selected operation will be performed on uploaded file.
Normal Flow	1. User opens the application. 2. User selects the feature. 3. User selects the desired format. 4. The system requests permission to access files on the device. 5. User grants the permission. 6. User chooses the file from the device. 7. File is uploaded.
Alternative Flows	If user does not grant the permission to access the file, user will be redirected to the previous screen.
Business Rules	App must have storage permission to access the file.
Assumptions	File must be available to upload.

2.5.8 Use Case 8

Table 23 Use Case 8

Use Case ID	UC-8
Use Case Name	Download File
Actors	End User
Description	This use case allows the user to download the processed files such as compression, conversion, text extraction and pdf files.
Trigger	User selects the “Download” option after the completion of file operation.
Pre-Conditions	The requested file must be successfully processed and available for

	download.
Post Condition	The file is successfully downloaded and saved to device storage.
Normal Flow	1. User completes an operation. 2. User selects the "Download" option. 3. System downloads and saves the file to device storage. 4. User receives confirmation of successful download.
Alternative Flows	If an error is occurred during download, user is notified.
Business Rules	• Only successful files can be downloaded. • Device permission is required.
Assumptions	• The user has sufficient device storage. • Device permissions are granted.

2.5.9 Use Case 9

Table 24 Use Case 9

Use Case ID	UC-9
Use Case Name	Share File
Actors	End User
Description	This use case allows the user to share the processed files from history.
Trigger	User clicks "Share" option for a processed file in history.
Pre-Conditions	The selected file must exist and be available for sharing.
Post Condition	File is successfully shared.
Normal Flow	1. User opens the application. 2. User access history items.

	3. History displays last 5 downloaded files. 4. User selects the file; user wants to share. 5. User clicks “Share” option. 6. System opens the available platform. 7. User selects the preferred sharing method. 8. System attaches the file. 9. User confirms sharing. 10. File is shared successfully.
Alternative Flows	If file is unavailable, sharing is cancelled.
Business Rules	• Only downloaded files can be shared. • User permissions are required for sharing access.
Assumptions	• File exists and is accessible. • Device permissions are granted.

2.5.10 Use Case 10

Table 25 Use Case 10

Use Case ID	UC-10
Use Case Name	Delete File
Actors	End User
Description	This use case allows the user to delete the processed files from history.
Trigger	User clicks “Delete” option for a processed file in history.
Pre-Conditions	File must exist in the application.
Post Condition	File is removed from the database.
Normal Flow	1. User opens the application. 2. User access history items.

	3. History displays last 5 downloaded files. 4. User selects the file; user wants to delete. 5. User clicks “Delete” option. 6. File is deleted successfully from the database.
Alternative Flows	If user clicks the cancel, file is not deleted.
Business Rules	• File is removed permanently. • Confirmation is required before deletion.
Assumptions	• File exists and is accessible. • Device permissions are granted.

2.5.11 Use Case 11

Table 26 Use Case 11

Use Case ID	UC-11
Use Case Name	Rename File
Actors	End User
Description	This use case allows the user to rename an existing file.
Trigger	User clicks “Rename” option.
Pre-Conditions	The file must be available in the application.
Post Condition	The file name is successfully updated in the database. The updated file name is reflected in the history.
Normal Flow	1. User opens the application. 2. User access history items. 3. History displays last 5 download files. 4. User selects the file; user wants to rename.

	5. User selects “Rename” option and a text box is shown. 6. User writes the new name in the text box. 7. User clicks the “Rename” button. 8. File successfully renamed.
Alternative Flows	If user clicks the cancel, file is not renamed.
Business Rules	File name must not be empty.
Assumptions	• File exists and is accessible to rename. • Device permissions are granted.

2.5.12 Use Case Diagram

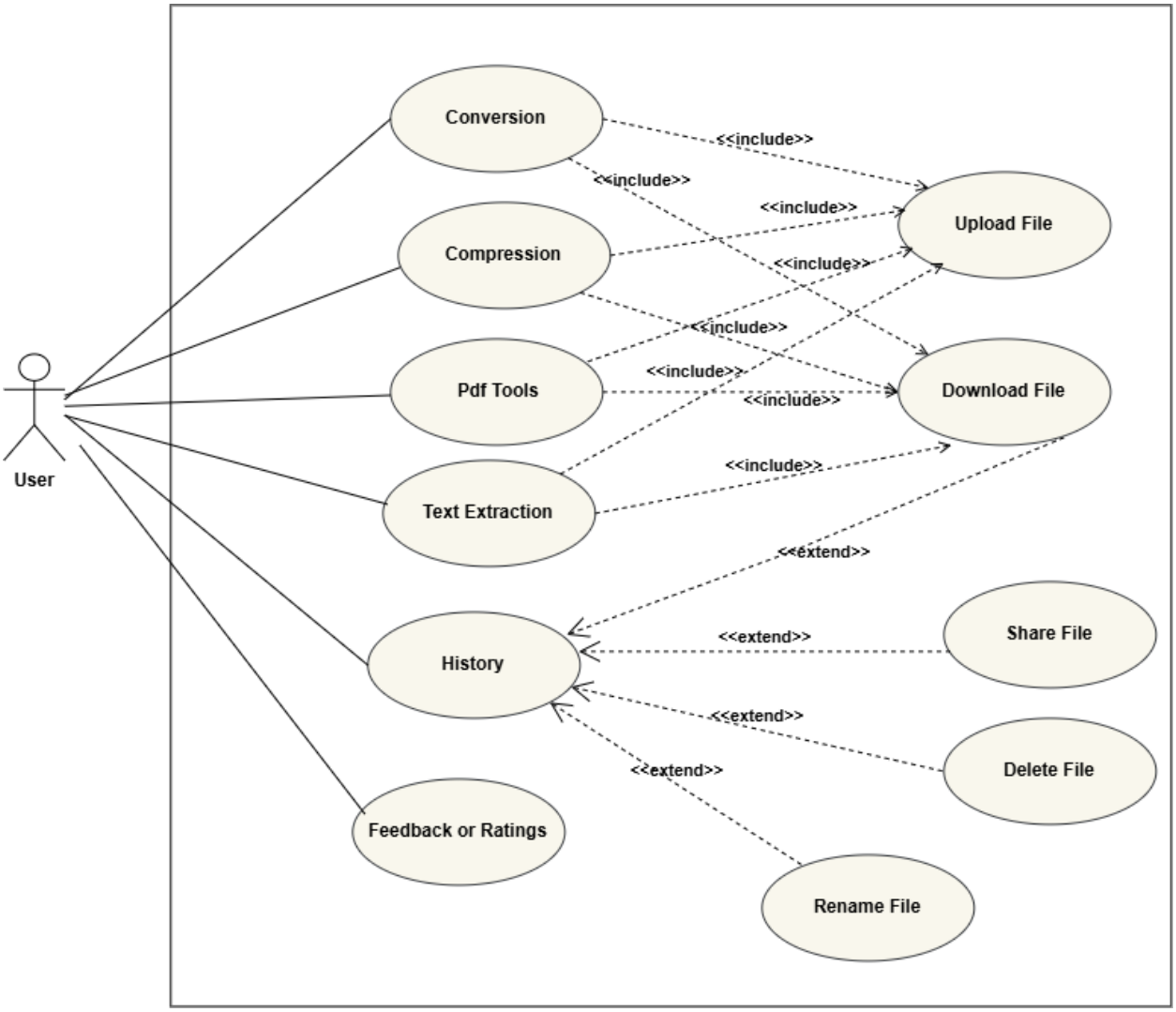


Figure 2-1 Use Case Diagram

2.6 External Interfaces

This section describes the external interface requirements of the Hybrid Format Converter, including user, hardware, software, and communication interfaces necessary for proper system interaction and operation.

2.6.1 User Interface Requirements

1. **General UI Requirements**
   - The system shall provide a simple and intuitive UI.

- The system shall be responsive to different screen sizes of mobile and tablets.
- The system shall follow the consistent design, including layouts and readable font size.
- The buttons such as “Convert”, “Compress”, “Extract”, “Submit” and “Delete” are available.

2. **Applicant Interface Requirements**
   - The system shall provide an interface that allows the user to select a file from the device for processing.
   - The system shall include the download screen to allow the user to download the processed file.
3. **Navigation and Messages**
   - The system shall have different navigation options like home, history and settings.
   - Different messages such as “Feedback Submitted” and “File Downloaded” will be shown on screen.
   - Error messages such as “Conversion failed” and “Compression Failed” will be shown on screen in case of any unsuccessful file operation.

2.6.2 Software Interfaces

1. **Database System**
   - Room Database is used which is a layer on SQL lite to store last 5 downloaded files.
   - Firebase Firestore is used which is a no SQL based database to store the feedback of users.
2. **Technologies**
   - It is an android application which will be developed using XML for layout and app logic will be implemented using Java.
3. **External Services**
   - Firebase Firestore will be used to store the feedback of users.
   - External API’s will be used for advanced conversion formats and online text extraction.
   - The system will integrate different libraries such as FFmpeg, Pdf box and OCR Space to perform offline file conversion, compression and online text extraction.

2.6.3 Hardware Interfaces

- The system will work properly on android 7.0+ versions.
- The system will depend on device processing unit (CPU) and memory to perform offline processing.

- The system will take input such as upload file and submit feedback through the touch screen.
- Display screen is required to view the output.

2.6.4 Communication Interfaces

1. **Network Requirements**
   - The system should primarily function offline.
   - Internet connection is required for submission of feedback and advance features.
2. **File Sharing**
   - The system shall allow the user to share the converted files through email and any other messaging application.
3. **Messages**
   - Different messages such as “Feedback Submitted” and “File Downloaded” will be shown on screen.
   - Error messages such as “Conversion failed” and “Compression Failed” will be shown on screen in case of any unsuccessful file operation.

2.7 Summary

This chapter outlines the process of identifying, eliciting, and analyzing system requirements. Users are divided into two main categories. To gather requirements, techniques such as open surveys, observation, and use case analysis are applied. The collected requirements are then classified into functional and non-functional types, supported by detailed use case descriptions and diagrams.

Chapter 3

Design and Architecture

3. System Design

System design elaborates product perspective in which platform will operate including dependencies and interaction with other systems. It also describes the design constraints.

Product perspective

The hybrid Format Converter is an android application that enables the user to convert the formats of their files with or without internet connection. It also provides PDF tools, compression and text extraction. This application can be used by teachers, students and office workers for work or education purpose. Firebase Firestore will be used to store the feedbacks given by users.

Dependencies & Interaction

- The application will work on android 7.0+ versions.
- The user needs the RAM of at least 3GB and free storage of 500 MB.
- The system will interact with external APIs to perform online conversions and text extraction.
- Our system will interact with firebase Firestore for storing feedbacks.

Design Constraints

- Usability: Application must be simple, clean with intuitive UI.
- Security: Application should access the file only when permission is granted by user.
- Responsiveness: Messages such as feedback submitted and file downloaded must appear on screen within an acceptable time to avoid any inconvenience.
- Availability: Cloud features like API calls for online functionality and firebase Firestore should work properly once internet connection is restored after disruption

3.1 Design Considerations

3.1.1 Assumption and dependencies

- The app must support offline functionality.
- For online conversions, stable internet is required.
- The application will work on android 7.0+ versions.
- The user needs the RAM of at least 3GB and free storage of 500 MB.

3.1.2 Limitations:

- It is an android application that means it will only work on an android device neither on iOS nor on desktop.
- To perform online conversions, user needs a stable internet.

- All existing formats for conversion will not be included in this app.
- All provided conversions will not be offline.
- Compression feature will not support all formats.

3.1.3 Risks

- Limited availability of formats for conversion, compression, text extraction and PDF Tools could affect the usability of our system.
  Mitigation: We ensure feedback mechanism so user can state their required format.
- Unstable internet connection can affect the performance of online conversions.
  Mitigation: We ensure maximum conversions without internet connection.
- Our application has limited PDF tools that can limit the functionality.
  Mitigation: More tools would be added on user demand.

3.2 Design Models

Design Models include a set of diagrams that represent system's behavior and structure. It includes Use Case, Class, sequence and state Diagrams showing relationships and interactions. These models are used to illustrate different perspective of application, ensuring clarity.

3.2.1 Class Diagram

A class diagram is a visual representation of the structure of the system, showing its classes, attributes, methods and relationships between them.

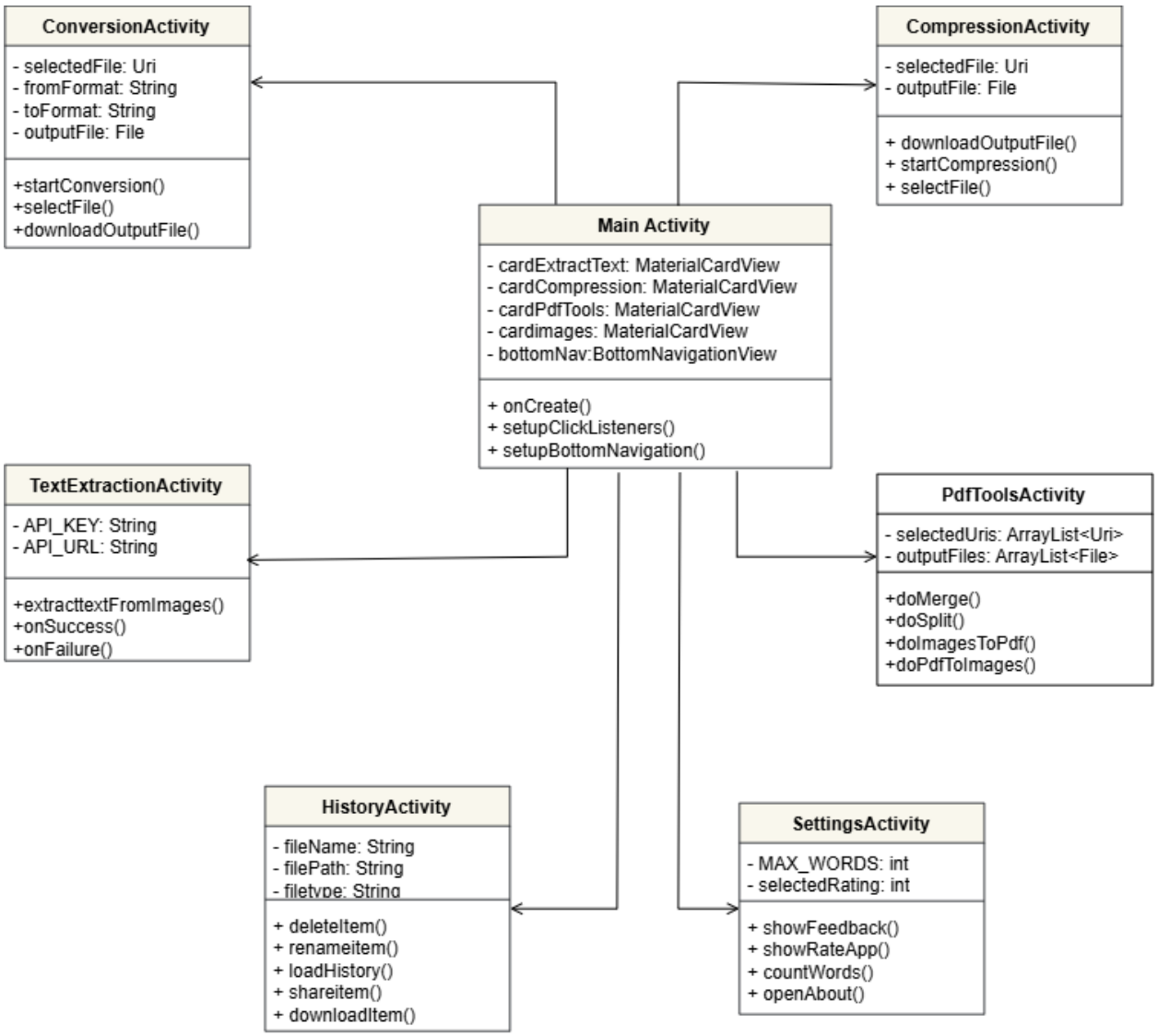


Figure 3-1 Class Diagram

Description

It represents the main classes of Hybrid Format Converter, their attributes, methods and relationship. This diagram represents different classes such as Main Activity, ConversionActivity, CompressionActivity, TextExtractionActivity, HistoryActivity, SettingsActivity and pdfToolsActivity and each class has its own attributes and function. Each class shows a relation with other class.

3.2.2 Sequence Diagram

It illustrates how the user interacts with app to perform file operation. It tells about the sequence of messages exchanged, with collaboration of different components to perform a task.

These sequence diagrams show how our hybrid converter handles both offline and online processing. The End User interacts with the Application UI to select formats and upload a file, which is then sent to the respective module. For offline mode, the module processes the file locally and returns the result, while for online mode, the request is sent to cloud services and the converted file is returned. After processing, the result is displayed to the user and saved in the Room Database as history, followed by a successful download.

Conversion Module

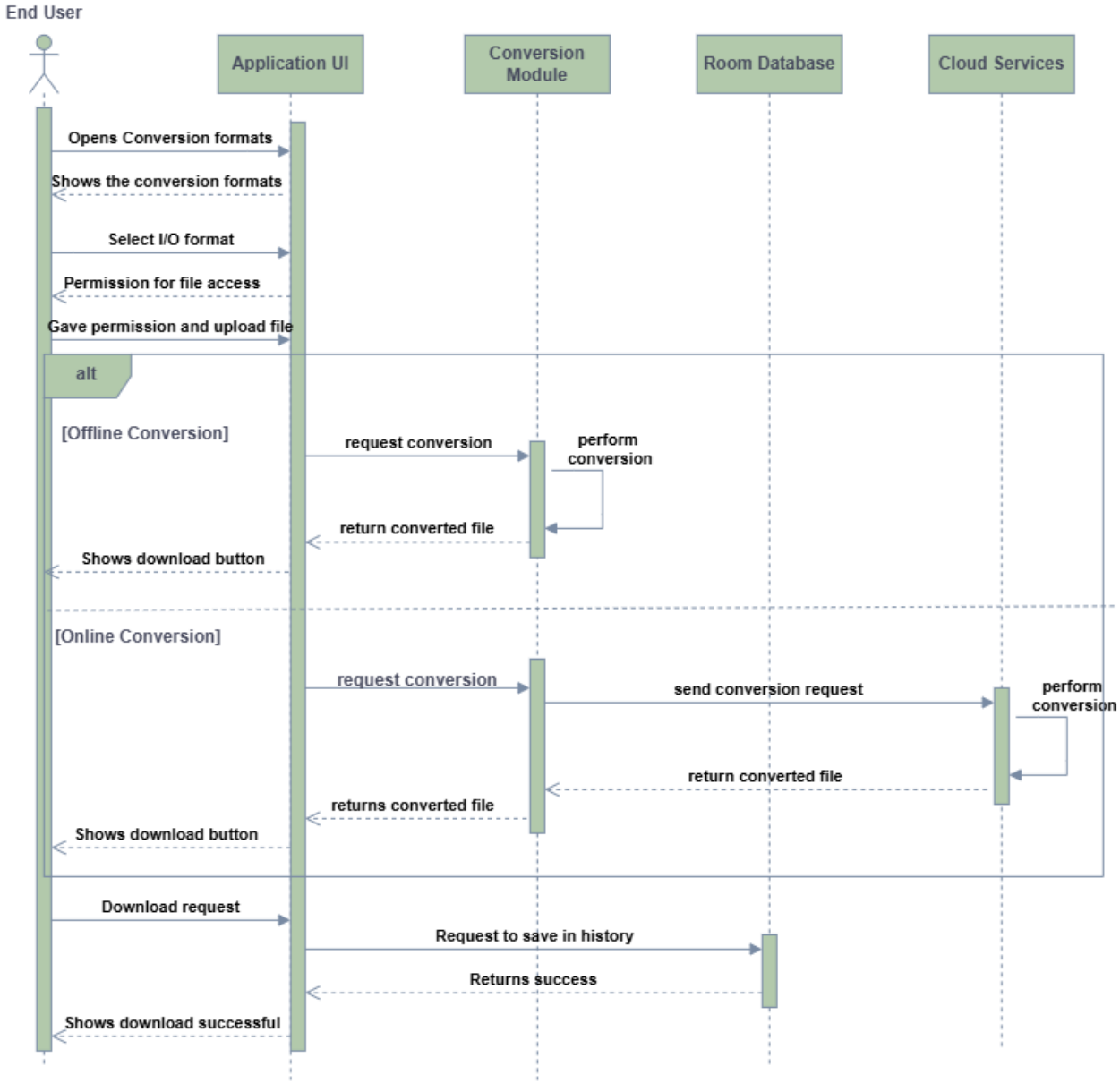


Figure 3-2 Sequence Diagram for Conversion Module

Compression Module

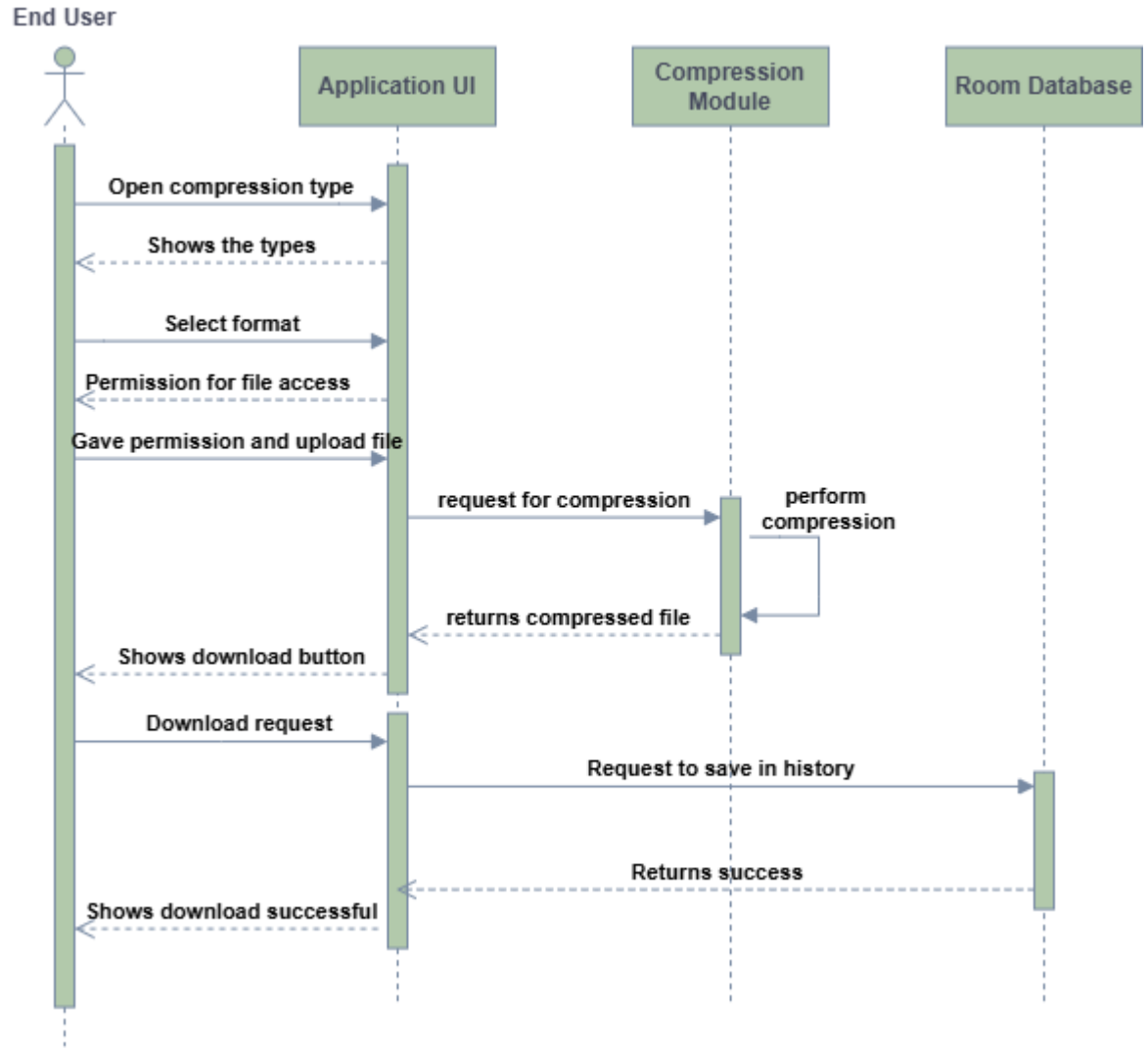


Figure 3-3 Sequence Diagram for Compression Module

Text Extraction Module

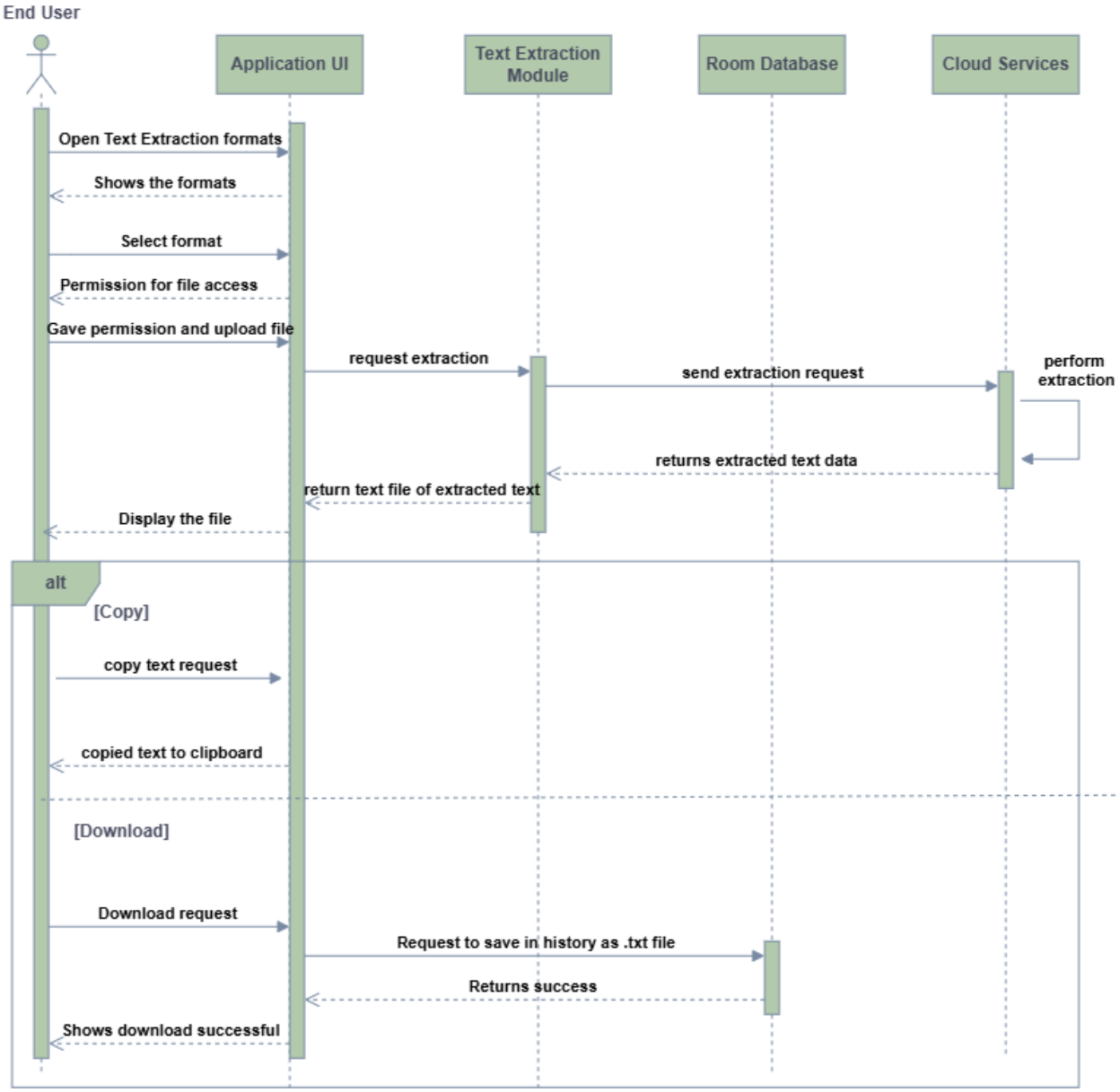


Figure 3-4 Sequence Diagram for Text Extraction Module

Pdf Tools Module

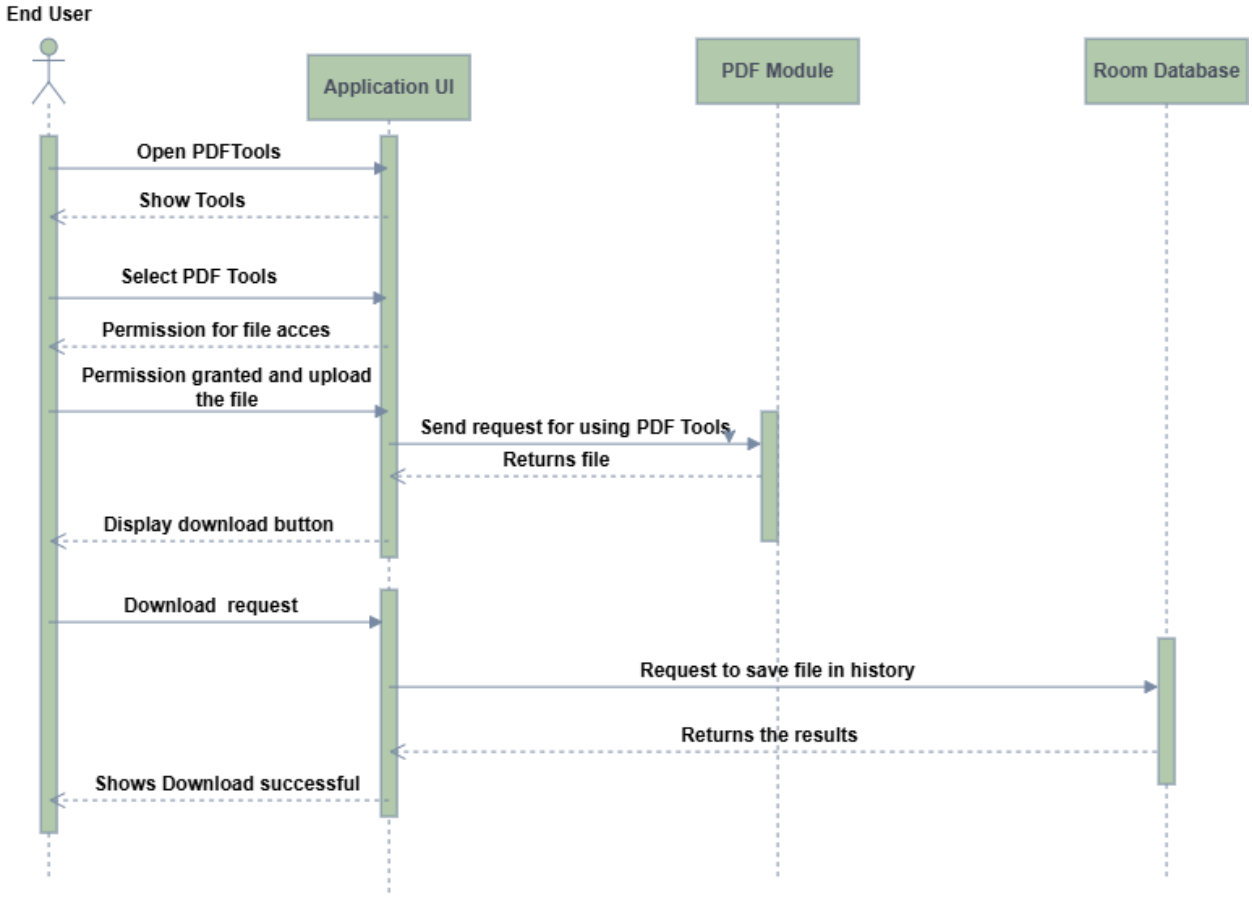


Figure 3-5 Sequence Diagram for Pdf Tools Module

History Module

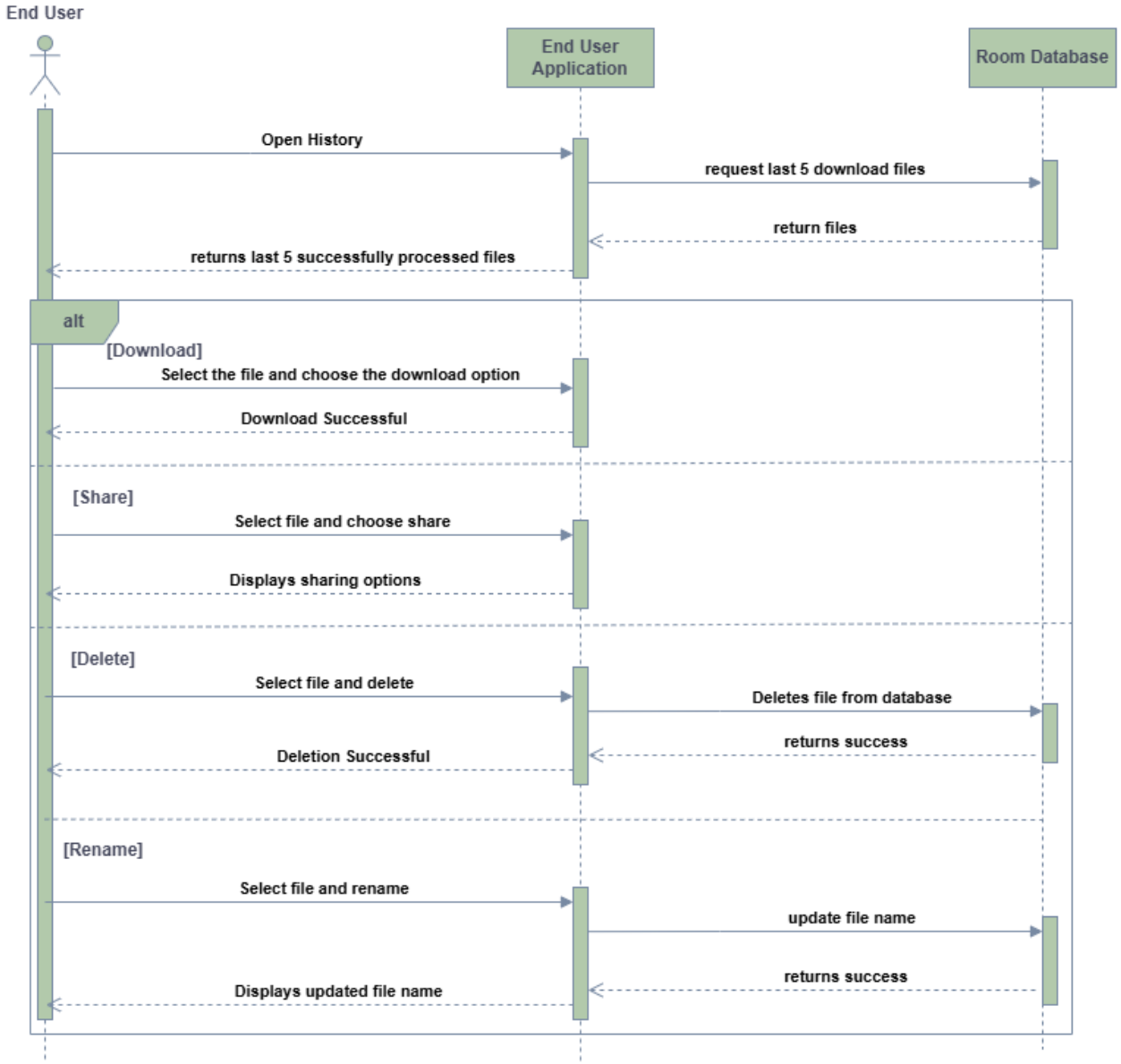


Figure 3-6 Sequence Diagram for History Module

Feedback Module

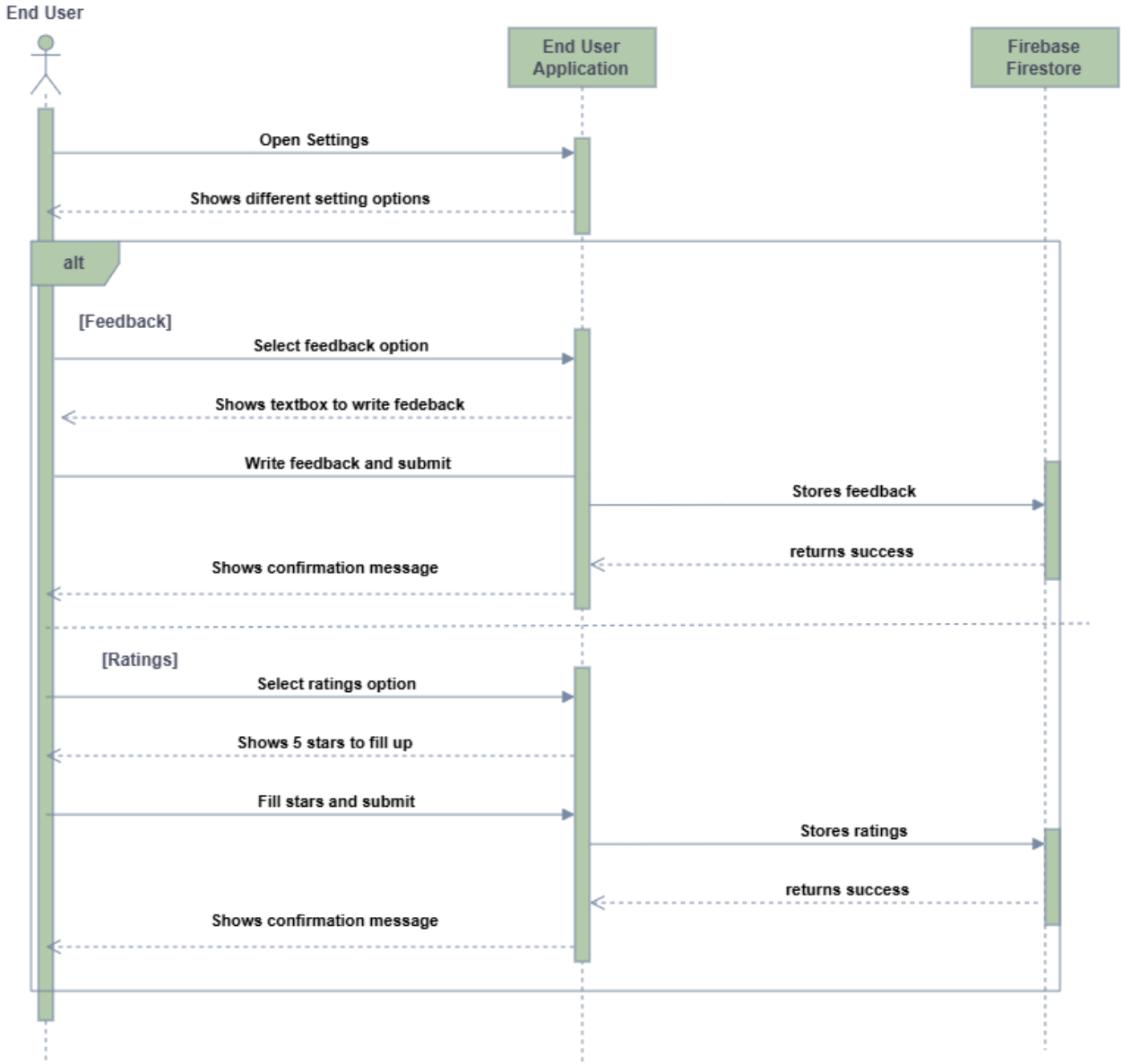


Figure 3-7 Sequence Diagram for Feedback Module

3.2.3 State Transition Diagram

It is used to represent the different states of our Hybrid Converter app while performing any operation. It represents different states of the system like displaying home screen, uploading a file, downloading a file, and how user moves from one state to another. It starts with user selecting a file operation following with granting permission for uploading the file, after operation ends user can download the file.

Conversion Module

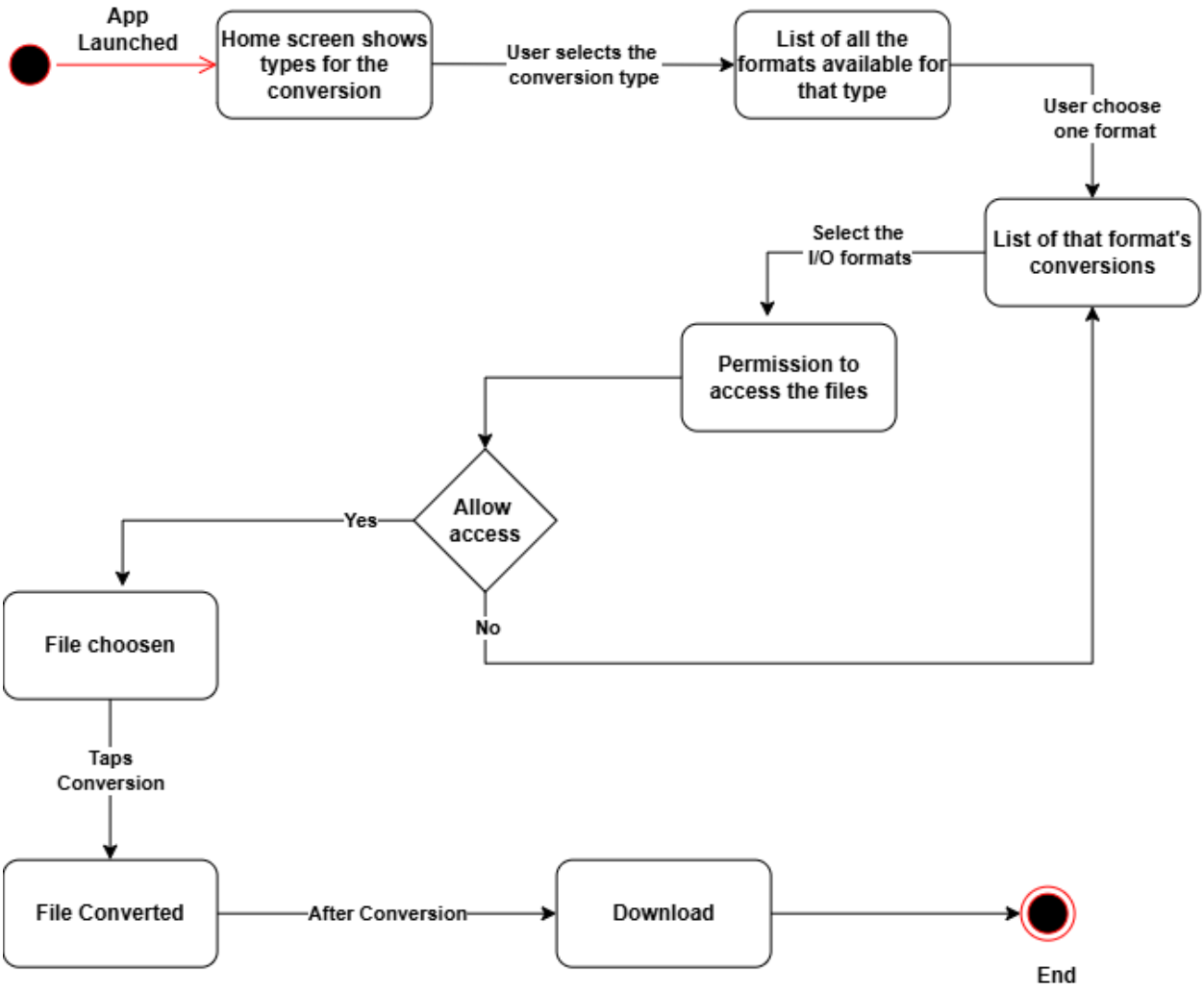


Figure 3-8 State Diagram for Conversion Module

Compression Module

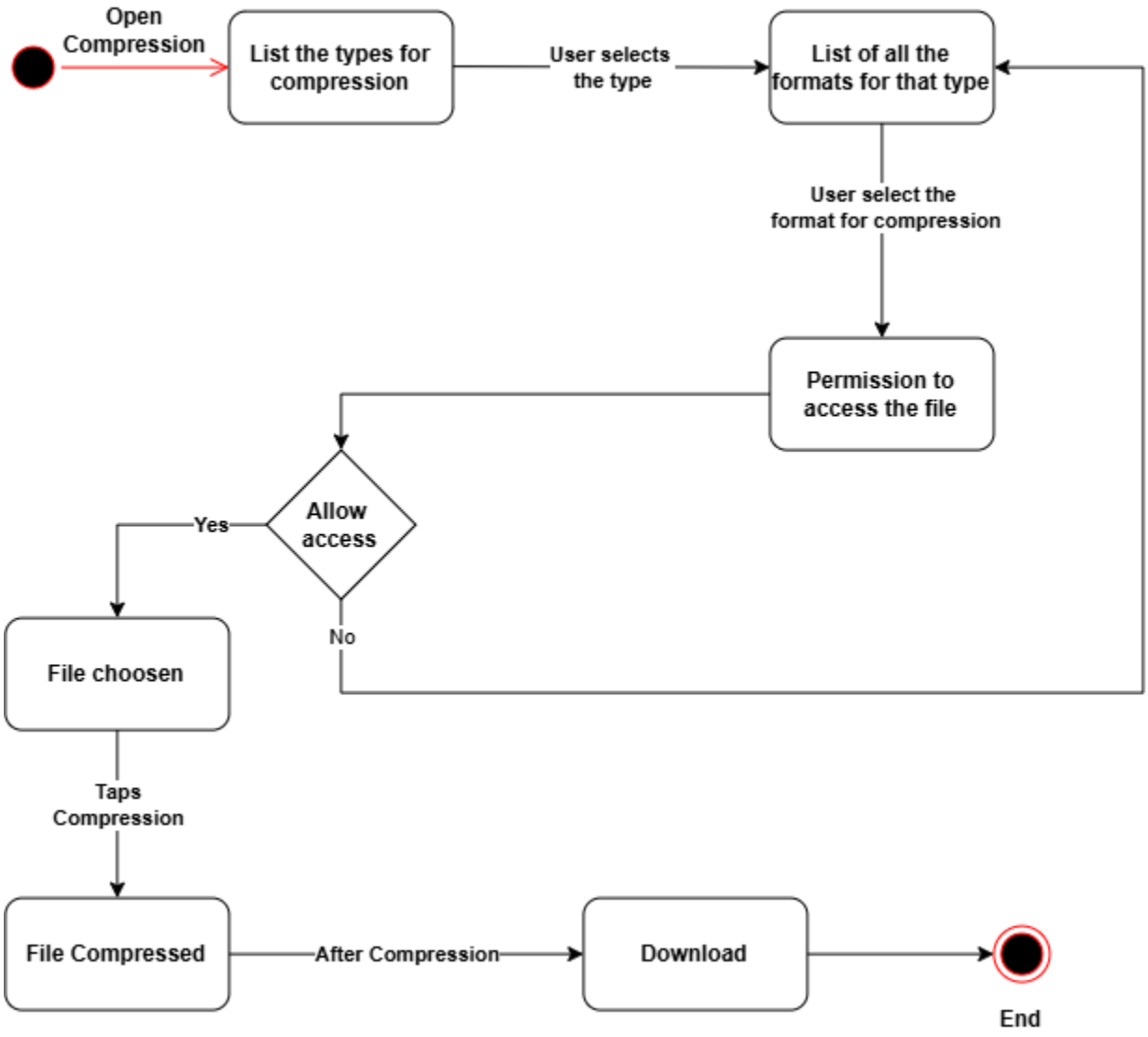


Figure 3-9 State Diagram for Compression Module

Text Extraction Module

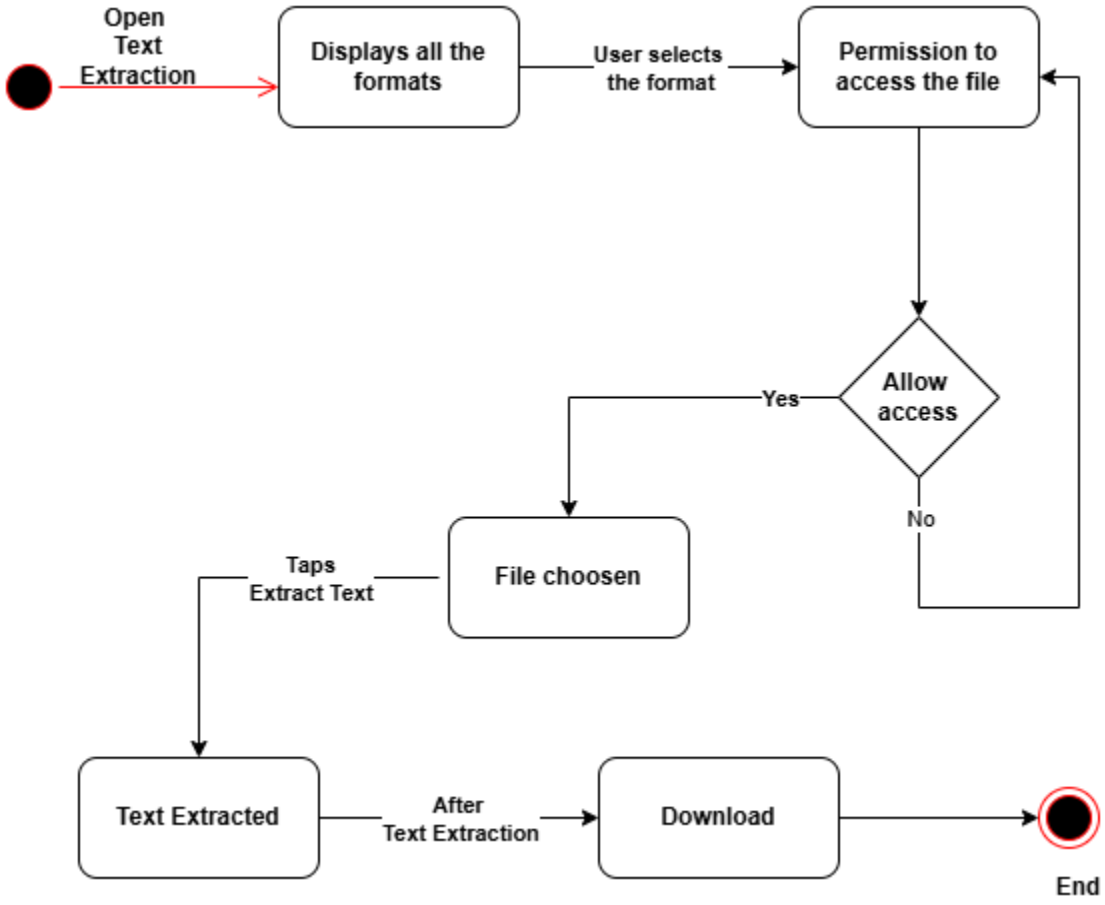


Figure 3-10 State Diagram for Text Extraction Module

PDF Tools Module

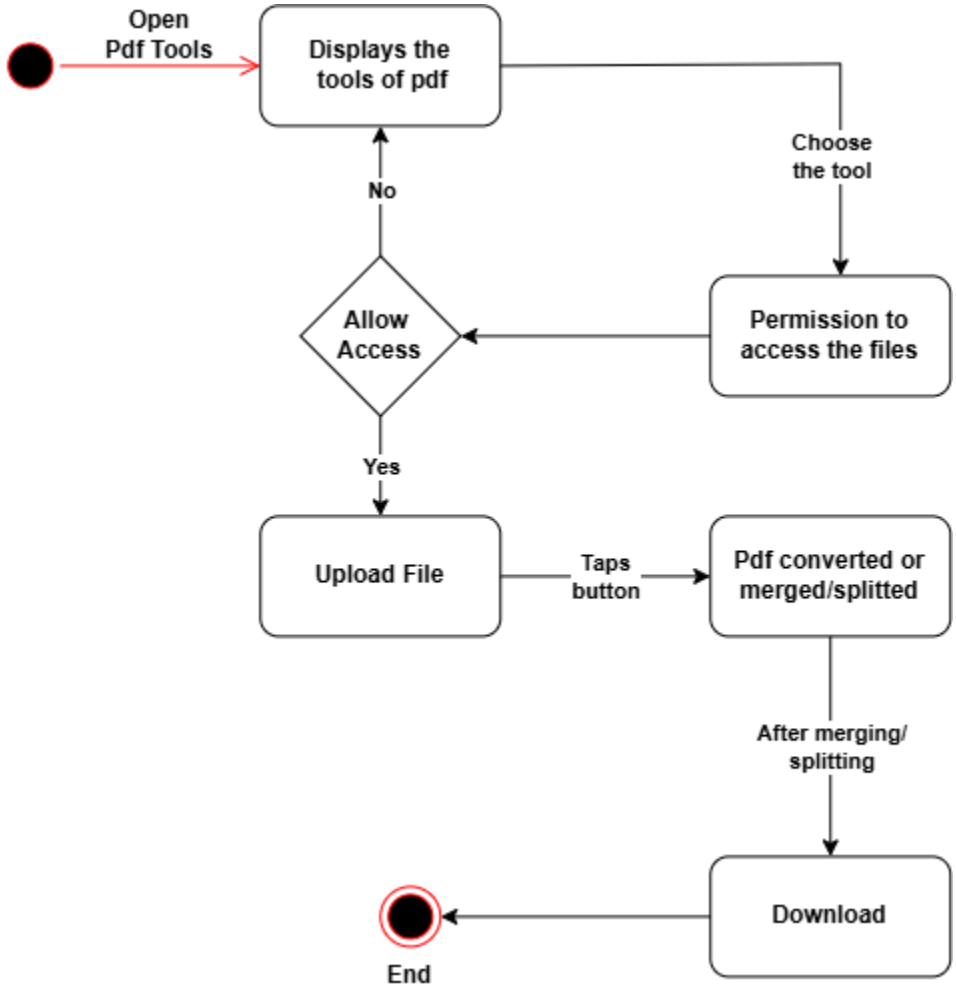


Figure 3-11 State Diagram for Pdf Tools Module

Feedback Module

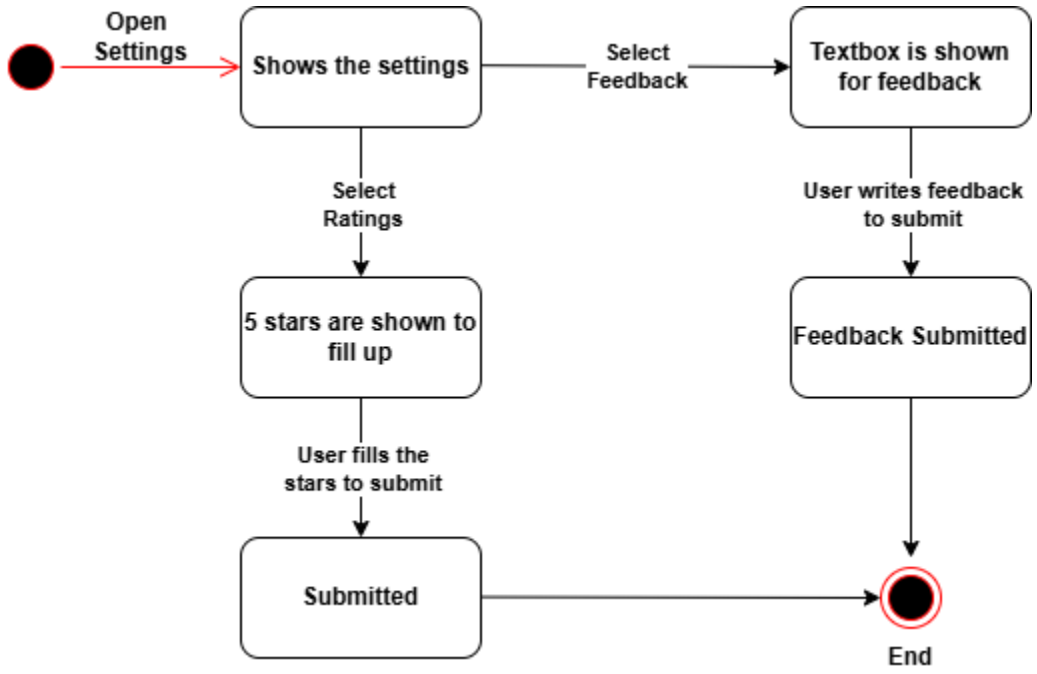


Figure 3-12 State Diagram for Feedback Module

History Module

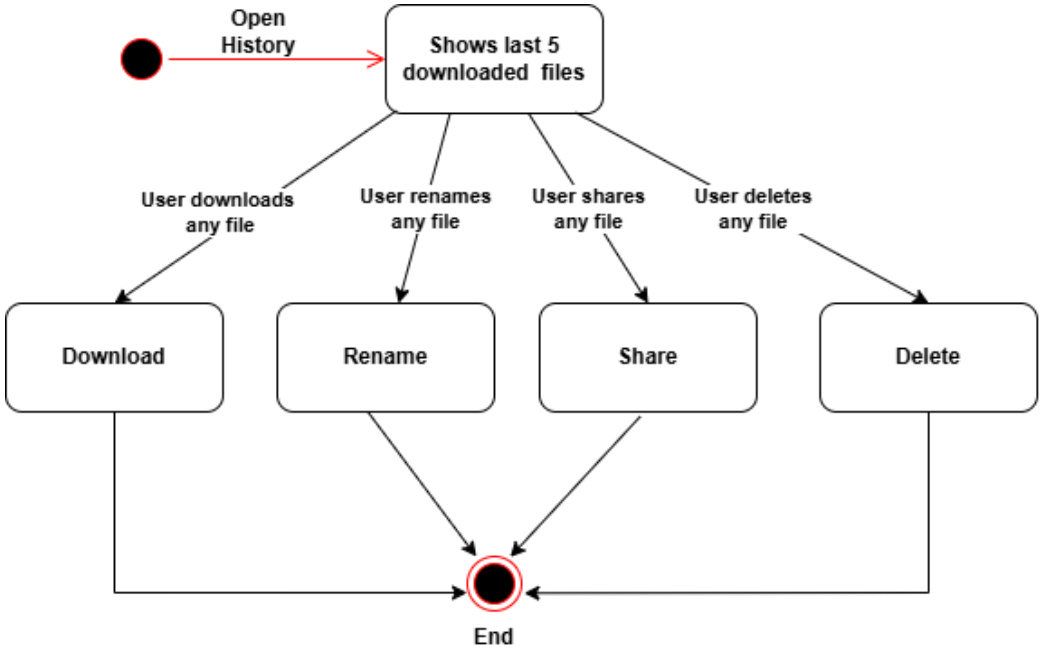


Figure 3-13 State Diagram for History Module

3.3 Architectural Design

It represents the major layers of system and how they interact to provide the system's functionality. It is concerned with designing the overall structure of the system. It includes Class, Component and Architecture Diagram.

- **Class Diagram**

  It represents the main classes their attributes, methods and relationships. It is already discussed above in Design Model Section.

3.3.1 Component Diagram

It represents major components of system and their interaction with each other.

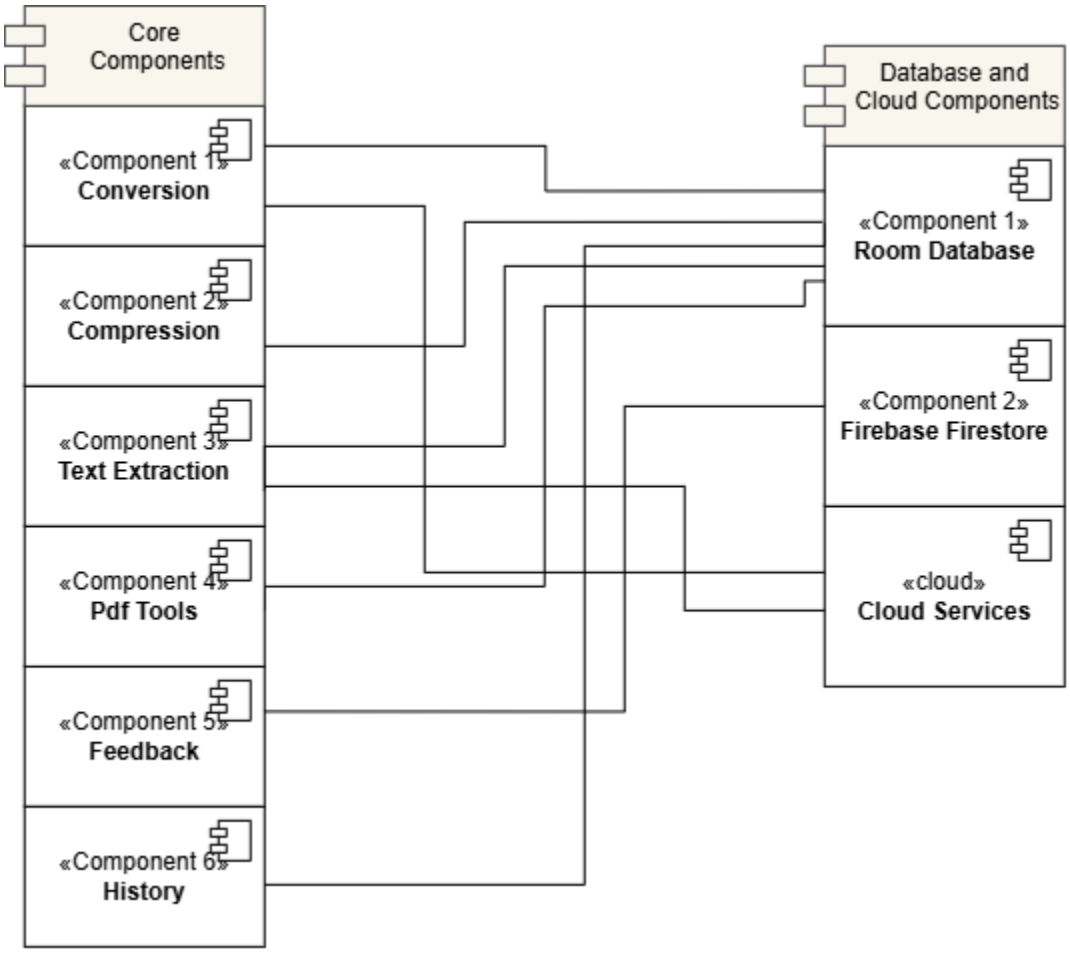


Figure 3-14 Component Diagram

Description

Core components such as Conversion, Compression, Text Extraction, PDF Tools, Feedback, and History handle the main functionalities of the system. These components interact with database

services like Room Database and Firebase Firestore to store and manage data. Additionally, cloud services support online conversion and text extraction.

3.3.2 Architectural Diagram

This project is using Model-View-View Model (MVVM) architecture pattern, which separates the UI, application's logic and data handling ensuring clarity and maintainability.

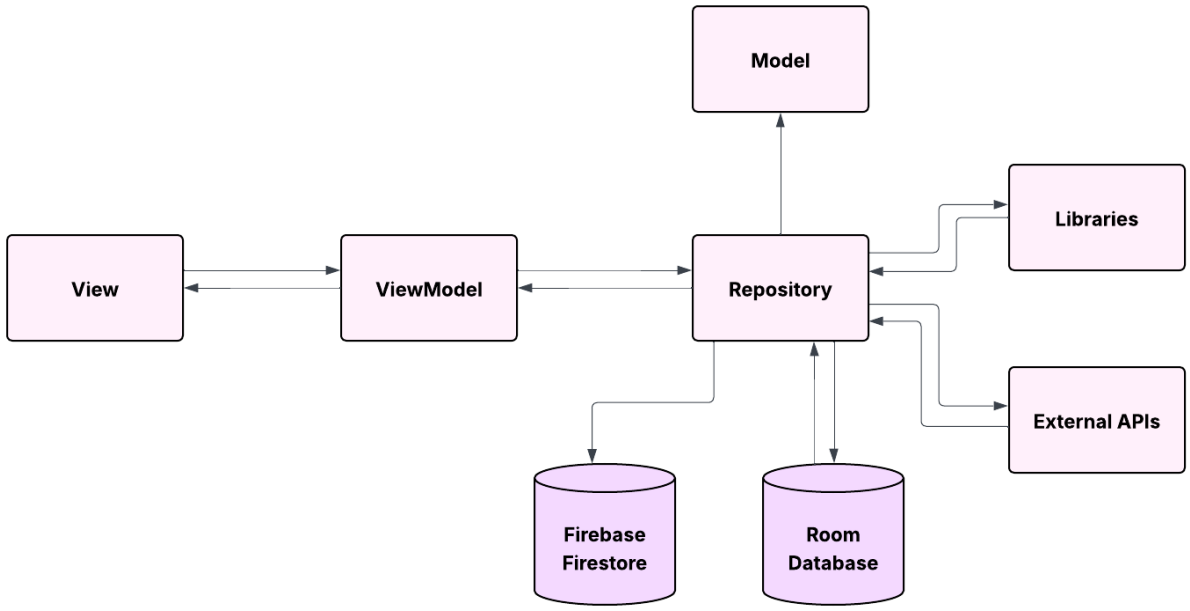


Figure 3-15 Architectural Diagram

Description

This diagram shows our hybrid converter using MVVM architecture. The View interacts with the ViewModel**,** which handles UI logic and communicates with the Repository**.** The Repository acts as the central data handler, fetching and storing data from sources like Firebase Firestore**,** Room Database, external APIs, and libraries. The Model represents the core data structure used across the system.

3.4 Data Design

Data Storage

For Database we are using Room Database, which is a layer on SQLite. It will store the converted files, compressed files, text extracted files and pdf files after performing file operations in history. Feedback will be stored in Firebase Firestore which a no SQL based database.

Data Processing

Conversion module is used for converting one file into other, compression module is used to reduce the sizes of files, text extraction module is used to extract the text, pdf tools are used to merge or split the pdf. The output files of all these modules are stored in history.

Data Organization

All of these files are organized in Room Database in the form of tables. Feedback Data is organized in the form of collections and documents in Firebase Firestore.

Table 27 Data Organization

Module	Database	Data Structure
Conversion	Room Database	Model classes, list, database table
Compression	Room Database	Model classes, list, database table
Text Extraction	Room Database	Model classes, list, database table
Pdf Tools	Room Database	Model classes, list, database table
History	Room Database	Model classes, list, database table
Feedback	Firebase Firestore	Model class, collection and documents

Entity Relation Diagram

ER diagram represents the structure of Hybrid Format Converter application and relationship between its major entities.

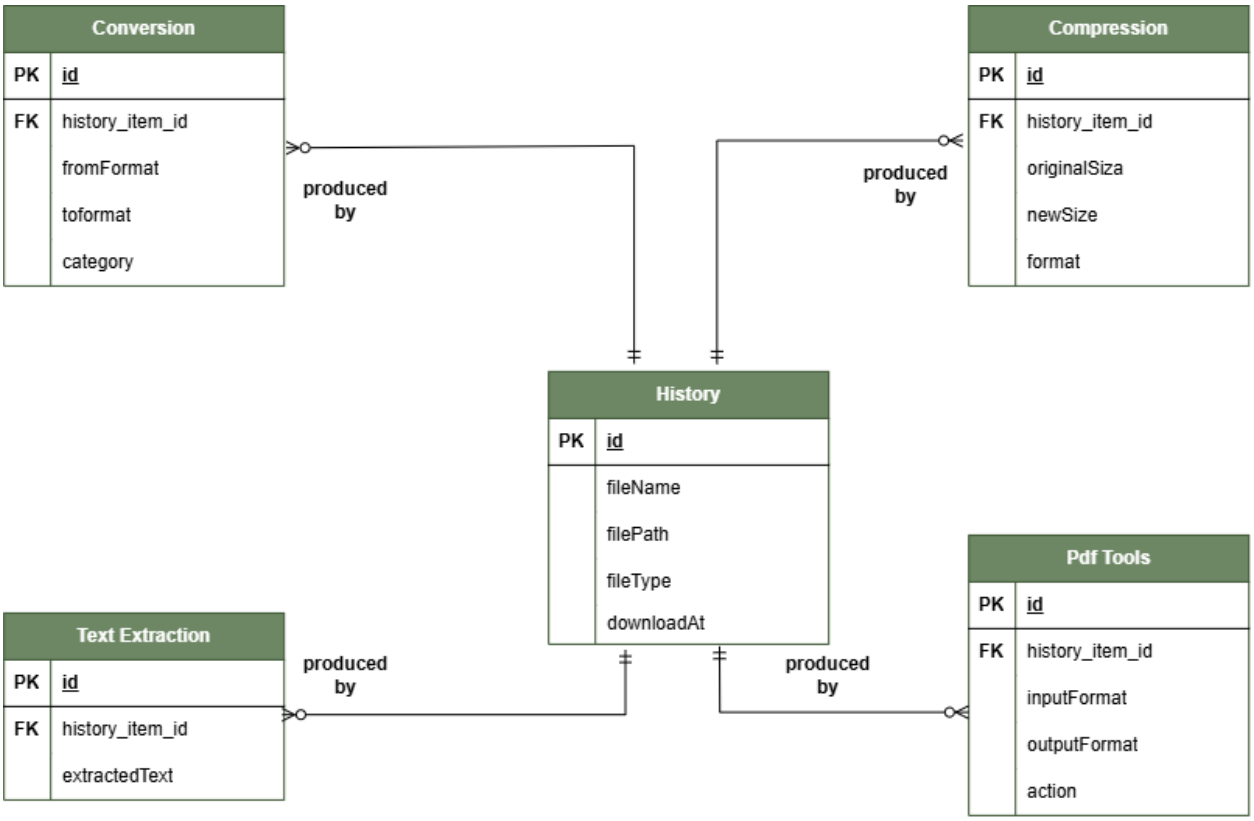


Figure 3-16 ER Diagram

Description

A file can undergo Conversion**,** Compression**,** Text Extraction, and PDF Tools processing, each stored in separate entities with their own attributes. Each of these operation tables references the History through a foreign key, showing that one file can have multiple processing activities.

3.4.1 Data Dictionary

Table 28 Data Dictionary

Terminology	Description
CompressionActivity	Class: Use to reduce the size of files of different formats.
ConversionActivity	Class: Use to perform the file conversion online as well as offline.
deleteItem()	Method: File can be deleted from the database by using this method.
downloadItem()	Method: Used to download the file after conversion, compression, text extraction and using pdf tools. It is also used to download the file from history.
doSplit()	Method: To split one pdf.
doMerge()	Method: To merge the pdfs.
FeedbackDialogHelper	Class: To allow user to submit feedback.
HistoryActivity	Class: Used to show the last 5 files that could be renamed, shared, download and deleted by user.
loadHistory()	Method: To load last 5 download file
PdfToolsActivity	Class: Used to handle pdf features such as pdf merge and pdf split.
RateAppDialogHelper	Class: To allow user to submit rating.
renameItem()	Method: Used to update the name of the file.
startCompression()	Method: To start the process of compression.
shareItem()	Method: Used to share the file.
showFeedback()	Method: Used to store feedback of user in Firebase Firestore.
selectFile()	Method: Used to select the file to perform chosen functionality.
TextExtractionActivity	Class: Handle the extraction of text online.

Structured Approach: Functions and Function Parameters

Table 29 Structured Approach

Functions with their parameters and data type	Description
loadHistory()	Loads last 5 files in history from database.
startConversion()	Perform conversion to selected format
startCompression()	Perform compression on uploaded file.
extractText(File pdfFile)	Extract text from non-selectable image.
doMerge(implementation-specific parameter)	Merge the selected pdfs.
doSplit(implementation-specific parameter)	Splits the uploaded pdf.
pdfToDocx(File imputFile,File outputFile)	To perform conversion of pdf to docx.
getMimeType(Uri uri)	Find out what type of file user selected.

- **Object-Oriented (OO) Approach: Objects, Attributes, Methods, & Method Parameters**

Table 30 Object Oriented Approach

Object	Attributes & their description
ConversionActivity	• Uri selectedFile: store selected file • File outputFile: store converted output file • String fromFormat: represent selected input file • String toFormat: represent selected output file
CompressionActivity	• Uri selectedFile: store selected file • File outputFile: store compressed output file
TextExtractionActivity	• String API_KEY: shows the API Key used for extraction • String API _URL: shows the tool we are using for extraction
PdfToolsActivity	• ArrayList<File> outputFiles: store the actual output files • ArrayList<Uri> selectedUris: store the location of files selected by users
HistoryActivity	• String filePath: represent file path. • String fileName: represent name of file.

FeedbackDialogHelper	• Int Max_Words: represent maximum words user can submit.

Objects	Methods & their parameters
ConversionActivity	• selectFile() • startConversion() • validateFile() • downloadOutputFile() • getMimeType(Uri uri)
CompressionActivity	• selectFile() • startConversion() • downloadItem()
TextExtractionActivity	• extractText(File pdfFile) • copyText()
PdfToolsActivity	• doSplit(implementation-specific parameter) • doMerge(implementation-specific parameter) • downloadItem(implementation-specific parameter)
HistoryActivity	• deleteItem(implementation-specific parameter) • renameItem(implementation-specific parameter) • shareItem(implementation-specific parameter) • downloadItem(implementation-specific parameter)
FeedbackDialogHelper	• showFeedback()

3.5 User Interface

The overview of our system's functionality is as following:

User's Perspective

From User perspective, app provide a simple UI that allow them to convert the formats of their files, compress their files, extract the text, use pdf tools, give feedback to the system and can access 5 last files in history.

User Workflow

Workflow begins by user selecting required file operation from all available tools such as conversion, compression, text Extraction and pdf Tools. User selects formats and uploads the file, after selecting the required feature, the system starts processing. After successful Conversion, compression, text extraction and pdf operation, a message will be displayed Conversion Successful, Compression Successful, Text Extracted, Pdf merged/split and no internet for online file conversion and extraction. User can give feedback by selecting feedback textbox in settings after completing the feedback user is shown a message Feedback Submitted.

Feedback Information

The application provides the necessary feedback information during workflow such as network error for online conversion, if internet is disrupted, File renamed, File downloaded, File deleted, Text Extracted, Conversion Completed, Compression Completed and Feedback Submitted.

Expected Feature

System enables the user to change the format of files, compress their files of different formats, extract the text, perform pdf operations, view history and give feedback of the system within responsible time.

3.5.1 Screen Images

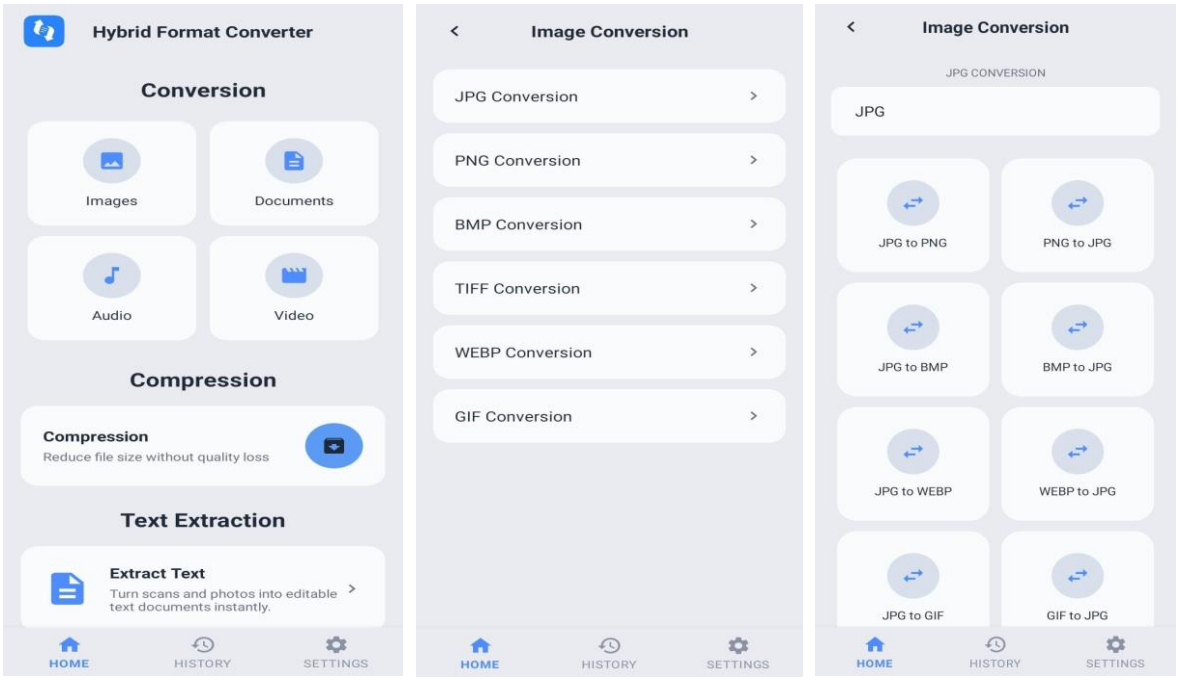


Figure 3-17 Screen Images

3.5.2 Screen Objects and Actions

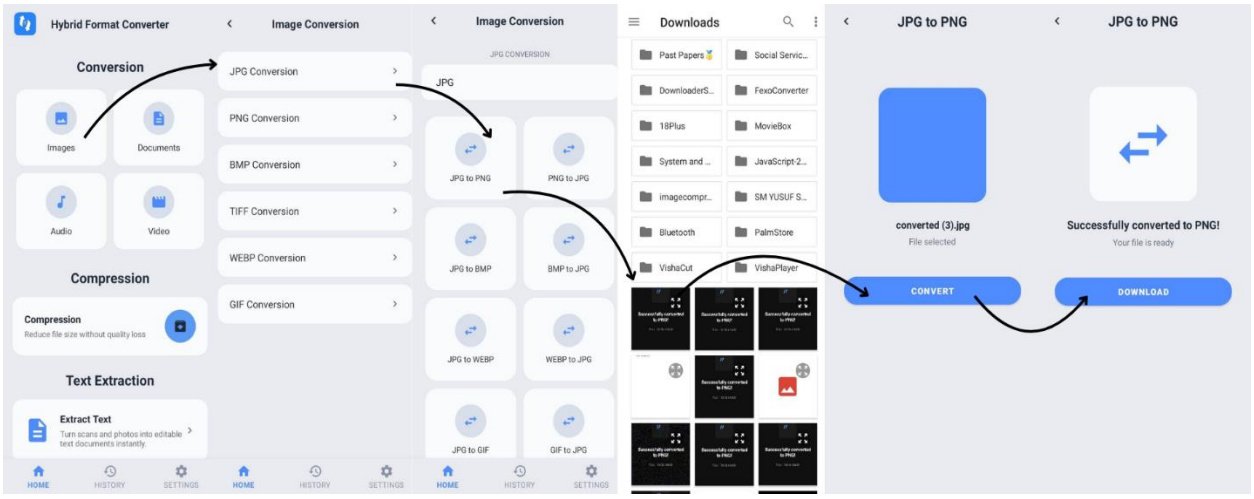


Figure 3-18 Screen Objects

3.6 Behavioral Model

The behavioral Model describes how the hybrid Format Converter interacts with user actions and how it transits between the different states.

Interaction Diagram

The interaction diagram shows how the user communicates with different modules of the Hybrid Format Converter system. The user initiates the process by selecting an operation such as conversion, compression, text extraction, or PDF tools and uploading a file. The respective module processes the request and returns the result to the user.

- **Sequence Diagram**
  It shows the step-by-step interaction between different objects of the system through message passing. Sequence Diagrams for each Hybrid Format Converter Modules are discussed above in Design Model Section.
- **State Transition Diagram**
  It illustrates the flow of different states of an object and transition between these states in Hybrid Format Converter to perform a specific task. State Diagrams for each Hybrid Format Converter Modules are discussed above in Design Model Section. However, an overall State diagram of Hybrid Format Converter is illustrated below

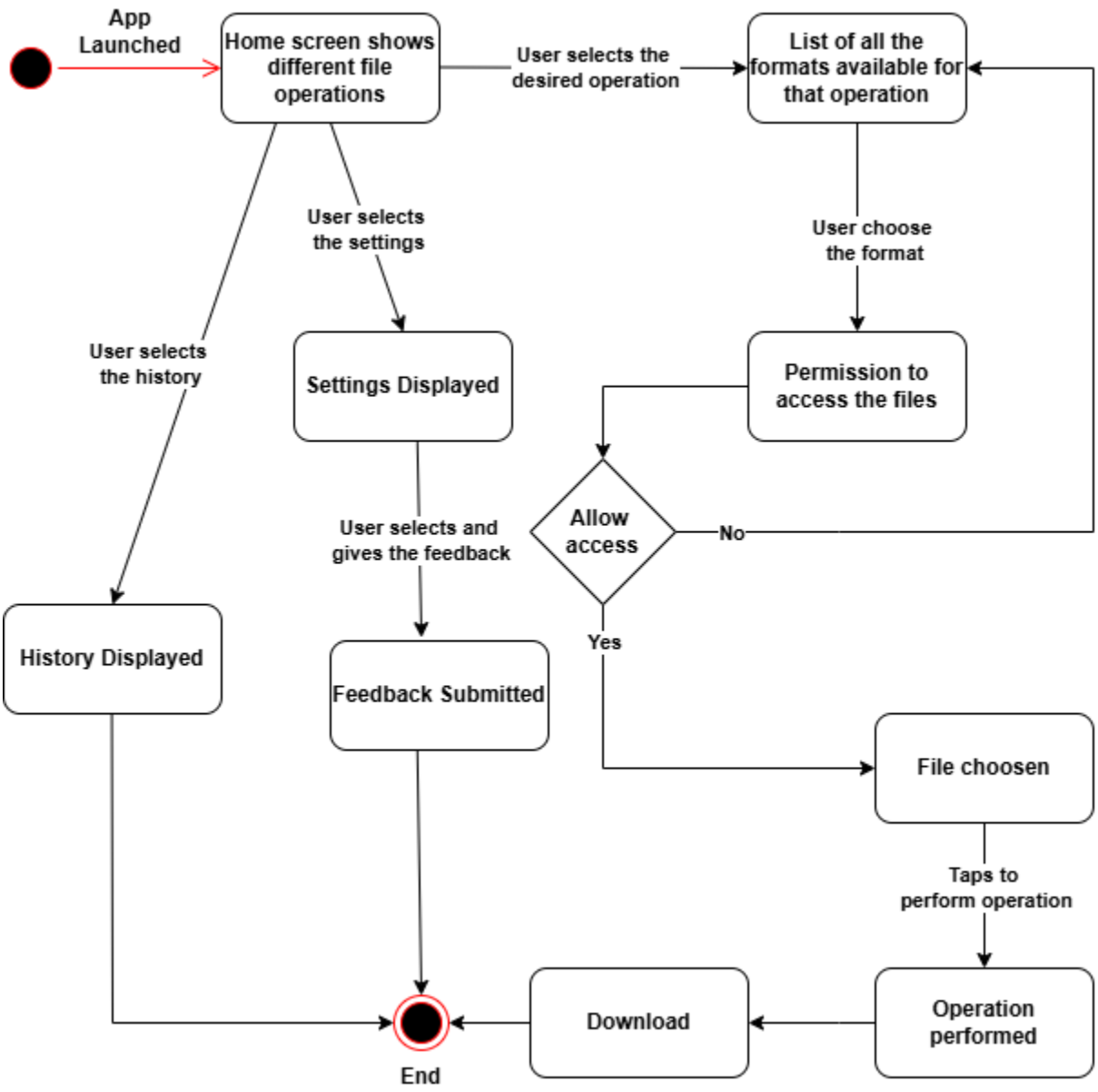


Figure 3-19 Overall State Diagram

3.7 Design Decisions

For Architecture, we used MVVM pattern because it's clean, organized, modular and easy to maintain. It also works great with Room Database as it is recommended architecture for android application by Google and also works best for multi-feature.
For Database, we use hybrid approach combining both Room database and Firebase Firestore. Room Database is SQL based database that is used to store history files and support file operations that do not require internet connection, Firebase Firestore is no SQL cloud database that will store feedback and ratings. It is compatible with android studio.
We are using Android studio for app development as it is official IDE for android app development. We are using java language because most of the applications are built in java.

3.8 Summary

This chapter presented detailed analysis of system design that includes architectural structure, sequence, state, component, class, Use Case diagrams. The MVVM architecture pattern is used because it is organized, clean and easy to maintain. This comprehensive analysis of system ensures that system meets its functional goals effectively.

Chapter 4

Implementation

4. Implementation

This chapter discusses the implementation details of the Hybrid Format Converter Android application. Each core feature is described as a separate algorithm using concise pseudocode, covering its workflow and logic.

4.1 Algorithms

Hybrid Format Converter uses twelve algorithms that together cover the full user workflow. Each algorithm below addresses one functional module or operation.

4.1.1 Algorithm 1 File Permission Request

Table 31 Algorithm 1

Algorithm	File Permission Request
Input	File operation request
Output	GRANTED / DENIED
Steps	function checkAndShow(context, actionLabel, callback 1. Get SharedPreferences "permission_prefs" 2. Check whether "allowed_always" is TRUE 3. If "allowed_always" is TRUE then 4. Call callback.onProceed() 5. return 6. Show permission dialog with 3 options: [Allow Access] [Only This Time] [Don't Allow] 7. If user selects "Allow Access" then 8. Save "allowed_always" as TRUE 9. Dismiss the dialog 10. Call callback.onProceed()

	11. If user selects "Only This Time" then 12. Do not save permanent permission 13. Dismiss the dialog 14. Call callback.onProceed() 15. If user selects "Don't Allow" then 16. Dismiss the dialog 17. Call callback.onDenied()

4.1.2 Algorithm 2 File Conversion

Table 32 Algorithm 2

Algorithm	File Conversion Offline (basic formats) and Online (advanced formats)
Input	Conversion category, fromFormat, toFormat
Output	convertedFile / Error Message
Steps	function startConversion(inputUri) 4. Show the Converting screen 5. Check the conversion category 6. If conversion is Image then 7. Build image conversion command using FFmpeg 8. Else if conversion is Audio then 9. Build audio conversion command using FFmpeg 10. Else if conversion is Video then 11. Build video conversion command using FFmpeg 12. Else if conversion is Document then convert using PDFBox and/or Apache POI 13. Perform the required document conversion 14. Generate the output document 15. Check whether conversion was successful

	16. If conversion is successful and output file exists then 17. Show the conversion completed screen

4.1.3 Algorithm 3 File Compression

Table 33 Algorithm 3

Algorithm	File Size Reduction Offline Compression
Input	Category
Output	Compressed file, original size, new size
Steps	function startCompression() 3. Get category, format, and input file URI 4. If any file information is missing then 5. Show "Missing file info" 6. return 7. Display selected file and load image thumbnail if category is Images 8. Show "Compressing..." progress state 9. Call CompressionRouter.compress() 10. If compression is successful then 11. Store the compressed output file 12. Calculate compression percentage 13. Display original size, new size, and saved percentage 14. Show compression result 15. Else 16. Display compression error message 17. Show error state

4.1.4 Algorithm 4 Text Extraction

Table 34 Algorithm 4

Algorithm	Text Extraction
Input	Context, mimeType
Output	ExtractedText / Error Message
Steps	function extractTextFromImage(context, imageUri, mimeType, callback) 1. Open and read the selected image 2. If image cannot be opened then 3. return "Unable to read image." 4. Create an OCR request with image data and settings 5. Send the request to OCR.Space API 6. If server response is unsuccessful or empty then 7. return "Server Error" 8. Parse the OCR response 9. If OCR fails or no text is detected then 10. return the appropriate error message 11. Extract and combine ParsedText from the results 12. If extracted text is not empty then 13. return extractedText 14. else 15. return "No readable text found."

4.1.5 Algorithm 5 Pdf Merge

Table 35 Algorithm 5

Algorithm	Merge PDF
Input	selectedUris

Output	Merged Pdf File/ Error Message
Steps	function doMerge() 1. create PDF merger 2. create output PDF file 3. add all selected PDF files to merger 4. merge all PDF files 5. add merged file to outputFiles 6. return merged file

4.1.6 Algorithm 6 Pdf split

Table 36 Algorithm 6

Algorithm	Split PDF
Input	selectedUris
Output	List of split pdf files / Error Message
Steps	function doSplit() 1. open selected PDF file 2. load PDF document 3. split PDF into individual pages 4. create separate PDF file for each page 5. add each split file to outputFiles 6. close PDF document 7. return split files

4.1.7 Algorithm 7 Download File

Table 37 Algorithm 7

Algorithm	Save processed file in device and in history
Input	filePath, filename
Output	History updated/ Error Message

Steps	function downloadItem(item) 1. get original file URI from history item 2. determine file MIME type 3. create a new file in Downloads using MediaStore 4. copy the file to Downloads 5. add the downloaded file to History 6. show "Saved to Downloads" 7. if download fails then delete incomplete download show "Download failed"

4.1.8 Algorithm 8 Share File

Table 38 Algorithm 8

Algorithm	Share File
Input	filePath, fileName
Output	Sharing options/ Error message
Steps	function shareItem(item) 1. get file URI from history item 2. determine file MIME type 3. create Share Intent with the file 4. grant temporary read permission 5. show sharing options to user 6. if sharing fails then show "Unable to share this file"

4.1.9 Algorithm 9 Delete File

Table 39 Algorithm 9

Algorithm	Delete File
Input	id, filename
Output	History item deleted/ Error message

Steps	function deleteItem(item) 1. show delete confirmation dialog 2. if user selects "Delete" then remove item from RoomDatabase 3. if deletion is successful then show "Removed from history" reload history 4. else show "Failed to delete"

4.1.10 Algorithm 10 Rename File

Table 40 Algorithm 10

Algorithm	Update File Name
Input	id, fileName
Output	Updated file name / Error Message
Steps	function renameItem(item) 1. get file name and separate name from extension 2. show rename dialog with current file name 3. user enters new name 4. if new name is empty then show "Name cannot be empty" return 5. combine new name with original extension 6. update file name in RoomDatabase 7. if successful then show "Renamed" reload history 8. else show "Rename failed"

4.1.11 Algorithm 11 Load History

Table 41 Algorithm 11

Algorithm	Load and Display Last 5 Successfully Processed Files
Input	Triggered when user navigates to History screen
Output	List of up to 5 recent files / Empty message
Steps	function loadHistory() 1. get history items from RoomDatabase through HistoryRepository 2. if history is empty then show "No recently processed files" hide history list return 3. display history items in RecyclerView 4. show history list

4.1.12 Algorithm 12 Feedback and Ratings Submission

Table 42 Algorithm 12

Algorithm	Feedback and Ratings Submission
Input	Triggered when user selects feedback or ratings
Output	Feedback submitted message / Error Message
Steps	function submitFeedback() 1. show feedback dialog with 200-word limit 2. count words as user types 3. if word count exceeds 200 then restore last valid text show "Maximum 200 words allowed" 4. user taps "Submit" 5. if feedback is empty then

	show "Please write your feedback before submitting" return 6. create feedback data with message, timestamp, app version, device and Android version 7. submit feedback to Firebase Firestore 8. if submission is successful then show "Thank you for your feedback!" close dialog 9. else show "Failed to send feedback. Please try again."

4.2 External APIs/SDKs

Table 43 External APIs/SDKs

Name of API and version	**Description of API**	**Purpose of usage**	**List down the API endpoint/function/class in which it is used**
FFmpegkit v6.1.1	Library used for the processing of audio, video and handle images.	Convert audio, video and image formats.	FFmpegKit.execute()
Apache PDFBox Android v2.0.27.0	A PDF processing library for creating, reading, and manipulating PDF documents on Android.	Used for PDF-related processing and manipulation	PDFMergerUtility.mergeDocuments() Splitter.split()
Apache POI	A Java library for	Used to read and	DocumentConverter

v5.2.5	working with Microsoft Office file formats.	process DOCX and other Office documents.	
Firebase Firestore - BOM v33.1.0	It is no SQL based database.	Used to store ratings and feedback.	FeedbackDialogHelper.show() collection("feedback").add()
AndroidX Room v2.6.1	A library that provides an abstraction layer over SQLite.	Used to store conversion history locally on the device.	HistoryRepository.addDownload() HistoryRepository.renameItem() HistoryRepository.deleteItem()
OCR.space API	An online Optical Character Recognition (OCR) API that extracts text from images.	Used to extract readable text from images when online OCR is required.	OcrHelper.extractTextFromImage()

4.3 Code Repository

Get Repository Link: https://github.com/Umama-Qamar/Hybrid-Format-Converter.git

It's a repository created for our Hybrid Format Converter, entire code is there and will be updated on daily basis as we test our app and improve it.

4.4 Summary

This chapter discussed the implementation details of the Hybrid Format Converter (HFC). Major algorithms with pseudocode covered core functions like file upload, conversion, compression, and text extraction. External APIs, SDKs, and libraries used in development were also discussed. GitHub repository management helped track progress and maintain the overall workflow of the system.

Chapter 5

Testing and Evaluation

5. Introduction

This chapter focuses on the testing and evaluation of our Hybrid File Converter project. It covers the validation and verification of different system components such as conversion, compression, text extraction, PDF tools, history and downloads features. The purpose of testing is to verify system accuracy, ensure that it meets user requirements, and identify any critical issues. A structured testing approach is followed, which includes unit testing, functional testing, integration testing, and performance testing.

5.1 Unit Testing (UT)

Unit testing is performed to verify functionality of each individual modules of the Hybrid Format Converter, ensuring that each feature works correctly in isolation. It ensures each unit is working correctly without depending on other.

5.1.1 Test Case 1

Table 44 Unit Test Case 1

Testcase ID	UT1
Requirement ID	R2
Title	Offline Conversion, Valid Input
Description	Test the startConversion() method with correct input format, input file path and output format without internet.
Objective	Verify that the file is successfully converted without the internet.
Driver/precondition	• No internet connection. • Valid input format exist in input file path.
Test steps	1. Enter the input file path. 2. Call the startConversion ().

	3. Verify that converted file is generated.
Input	selectedUris = [selected image Uri]
Expected Results	Converted file will be generated.
Actual Results	Pass
Remarks	Pass

5.1.2 Test Case 2

Table 45 Unit Test Case 2

Testcase ID	UT 2
Requirement ID	R2
Title	Online Conversion, Valid Input
Description	Test the startConversion () method with correct input format, input file path and output format with internet connection.
Objective	Verify that the file is successfully converted with the internet connection.
Driver/precondition	• Internet connection is needed. • Valid input format exists in input file path.
Test steps	1. Enter the input file path. 2. Call the startConversion (). 3. Verify that converted file is generated.

Input	selectedUris = [selected image Uri]
Expected Results	Converted file will be generated.
Actual Results	Pass
Remarks	Pass

5.1.3 Test Case 3

Table 46 Unit Test Case 3

Testcase ID	UT 3
Requirement ID	R3
Title	Compression, Valid Input
Description	Test the startCompression () method with correct input format, input file path and output format without internet connection.
Objective	Verify that the file is successfully compressed without the internet connection.
Driver/precondition	• Internet connection is not needed. • Valid input format exists in input file path.
Test steps	1. Enter the input file path. 2. Call the startCompression (). 3. Verify that converted file is compressed.
Input	selectedFileUri = valid image/file Uri, category = Images, format = JPG

Expected Results	Compressed file is returned.
Actual Results	Pass
Remarks	Pass

5.1.4 Test Case 4

Table 47 Unit Test Case 4

Testcase ID	UT 4
Requirement ID	R4
Title	Online Text Extraction, Valid Input
Description	Test the extractTextFromImage () method with correct input format, input file path with the internet connection.
Objective	Verify that text is extracted successfully with the internet connection.
Driver/precondition	• Internet connection is required. • Valid input format exist in input file path.
Test steps	1. Enter the input file path. 2. Call the extractTextFromImage (). 3. Verify that text is extracted and generate .text file.
Input	context = current Activity, imageUri = selected image Uri, mimeType = "image/jpeg"
Expected Results	Extracted text is returned.

Actual Results	Pass
Remarks	Pass

5.1.5 Test Case 5

Table 48 Unit Test Case 5

Testcase ID	UT 5
Requirement ID	R5
Title	Pdf merged, Valid Pdf files
Description	Test the doMerge () method with valid Pdf files.
Objective	Verify that Pdf is merged and returns a merged Pdf file.
Driver/precondition	Pdf files exist in input file path.
Test steps	1. Enter file path for pdf files. 2. Call doMerge (). 3. Verify that returned pdf file is merged and not null.
Input	selectedUris = [file1.pdf Uri, file2.pdf Uri]
Expected Results	Merged pdf is returned.
Actual Results	Pass
Remarks	Pass

5.1.6 Test Case 6

Table 49 Unit Test Case 6

Testcase ID	UT 6
Requirement ID	R5
Title	Pdf split, Valid Pdf files
Description	Test the doSplit () method with valid Pdf file.
Objective	Verify that Pdf is split and returns split Pdf files.
Driver/precondition	Pdf file exists in input file path.
Test steps	1. Enter file path for pdf file. 2. Call doSplit (). 3. Verify that returned pdf files is split and not null.
Input	selectedUris = [file.pdf Uri]
Expected Results	Split pdfs are returned.
Actual Results	Pass
Remarks	Pass

5.1.7 Test Case 7

Table 50 Unit Test Case 7

Testcase ID	UT 7

Requirement ID	R7
Title	Viewing History
Description	Test the loadHistory () function from the room database. Verify that only last 5 files are displayed.
Objective	Verify that the last 5 files are accessed from the room database without internet.
Driver/precondition	Internet connection is not needed.
Test steps	1. Insert some records in room database. 2. Call the loadHistory (). 3. Verify that the last 5 files are showing in history.
Input	Few records are inserted in room database.
Expected Results	Last 5 files will be available in history feature.
Actual Results	Pass
Remarks	Pass

5.1.8 Test Case 8

Table 51 Unit Test Case 8

Testcase ID	UT 8
Requirement ID	R6

Title	Feedback Submission, Valid message.
Description	Test feedback submission using FeedbackDialogHelper.show().
Objective	Verify that feedback is saved to Firebase Firestore with the internet and non-empty message.
Driver/precondition	• Internet connection is needed. • Firebase Firestore is connected.
Test steps	1. Call the FeedbackDialogHelper.show(). 2. Enter the feedback message. 3. Verify that “Feedback submitted” message is displayed.
Input	message=” Very helpful app”
Expected Results	Feedback is stored on Firebase Firestore and success message is displayed.
Actual Results	Pass
Remarks	Pass

5.1.9 Test Case 9

Table 52 Unit Test Case 9

Testcase ID	UT 9
Requirement ID	R2
Title	Conversion, Invalid Input

Description	Test the startConversion () method with correct input format and output format but wrong input file path
Objective	Verify that application shows error message.
Driver/precondition	Internet connection is required or not based on the advancement of conversion.
Test steps	1. Enter the wrong input file path. 2. Call the startConversion (). 3. Verify that system shows an error message.
Input	selectedUris = [invalid file Uri]
Expected Results	System shows an error message.
Actual Results	Pass
Remarks	Pass

5.2 Functional Testing (FT)

Functional testing was conducted to verify that each core feature of the Hybrid Format Converter system performs according to the specified functional requirements. This testing focused on validating user interactions, input processing, system responses, and expected outputs for major modules such as conversion, compression, text extraction, feedback and history management.

5.2.1 Test Case 1

Table 53 Functional Test Case 1

Testcase ID	FT 1
Requirement ID	R2

Title	Perform Conversion
Description	Test the complete conversion work flow from file uploading to file download.
Objective	Verify that the user can perform the conversions and it stored in the database.
Driver/precondition	• User has a file to upload. • Internet connection is required or not based on the advancement of conversion.
Test steps	1. Open the application and choose the type of conversion. 2. Select the format. 3. Grant the permission and upload the file. 4. Download the converted file.
Input	selectedUris = [selected file Uri]
Expected Results	Converted file is generated and displayed in history.
Actual Results	Pass
Remarks	Pass

5.2.2 Test Case 2

Table 54 Functional Test Case 2

Testcase ID	FT 2
Requirement ID	R3

Title	Perform Compression
Description	Test the complete compression work flow from file uploading to file download.
Objective	Verify that the user can perform the compression and it stored in the database.
Driver/precondition	• User has a file to upload. • Device has no internet connection.
Test steps	1. Open the application and choose the type of compression. 2. Select the format. 3. Grant the permission and upload the file. 4. Download the compressed file.
Input	selectedFileUri = valid image/file Uri, category = Images, format = JPG
Expected Results	Compressed file is generated and displayed in history.
Actual Results	Pass
Remarks	Pass

5.2.3 Test Case 3

Table 55 Functional Test Case 3

Testcase ID	FT 3
Requirement ID	R4

Title	Text Extraction
Description	Test the complete text extraction work flow from file uploading to file downloading a text file.
Objective	Verify that text is extracted and user can download .txt file.
Driver/precondition	A clear image is available.
Test steps	1. Open the application and choose extraction feature. 2. Select the format. 3. Grant the permission and upload the file. 4. Tap text Extract. 5. Text is extracted and .txt file is available for user to download.
Input	context = current Activity, imageUri = selected image Uri, mimeType = "image/jpeg"
Expected Results	Text is extracted and .txt file is displayed to download.
Actual Results	Pass
Remarks	Pass

5.2.4 Test Case 4

Table 56 Functional Test Case 4

Testcase ID	FT 4
Requirement ID	R5

Title	Pdf merged/pdf split
Description	Test the end-to-end pdf merged/split workflow followed by downloading the results.
Objective	Verify that pdf merged produce merged pdf and pdf split produce multiple pdfs.
Driver/precondition	Pdf files exist on device.
Test steps	1. Open pdf tools and select either pdf merge or pdf split. 2. Upload pdf file regarding operation 3. Tap pdf merge/split. 4. Resulted pdf/pdfs is displayed. 5. Tap download.
Input	Merge: selectedUris = [file1.pdf Uri, file2.pdf Uri] Split: selectedUris = [file.pdf Uri]
Expected Results	Merged pdf is generated. Split pdfs is split.
Actual Results	Pass
Remarks	Pass

5.2.5 Test Case 5

Table 57 Functional Test Case 5

Testcase ID	FT 5
Requirement ID	R7

Title	Viewing history
Description	Test that user can access previously processed files in history.
Objective	Verify the last 5 successfully processed files are displayed correctly.
Driver/precondition	User has at-least one previous record.
Test steps	1. Open application. 2. Navigate to history. 3. Access the saved records.
Input	Records are inserted in Room database.
Expected Results	Complete history of conversion, compression and text extraction are displayed.
Actual Results	Pass
Remarks	Pass

5.2.6 Test Case 6

Table 58 Functional Test Case 6

Testcase ID	FT 6
Requirement ID	R6
Title	Feedback Submission
Description	Test the complete text feedback work flow from writing feedback to

	submission.
Objective	Verify that a user can give feedback to the system and it is stored in Firebase Firestore.
Driver/precondition	• User has internet connection. • Firebase Firestore is connected.
Test steps	1. Open the settings and choose feedback. 2. Write the text feedback. 3. Submit the feedback and received a “Feedback Submitted” message.
Input	message=” very helpful app”
Expected Results	Feedback is stored in Firebase Firestore and success message is displayed.
Actual Results	Pass
Remarks	Pass

5.2.7 Test Case 7

Table 59 Functional Test Case 7

Testcase ID	FT 7
Requirement ID	R2
Title	Perform Conversion, invalid input
Description	Test the complete conversion work flow when user gives invalid input path.

Objective	Verify that application shows an error message.
Driver/precondition	Internet connection is required or not based on the advancement of conversion.
Test steps	1. Open the application and choose the type of conversion. 2. Select the format and grant the permission. 3. Upload the wrong input file path. 4. Verify that system shows an error message.
Input	selectedUris = [invalid file Uri]
Expected Results	System shows an error message.
Actual Results	Pass
Remarks	Pass

5.3 Integration Testing

Integration testing was performed to evaluate the interaction and coordination between different modules of the Hybrid Format Converter system. This phase focused on verifying that interconnected components such as user interface screens, backend services, Firebase integration, file processing modules, and database operations work together.

5.3.1 Test Case 1

Table 60 Integration Test Case 1

Testcase ID	IT 1
Requirement ID	R2
Title	Frontend →Backend →Room Database: Conversion

Description	Test the full integration path from uploading the file to using library or API to Room database storage.
Objective	Verify that the conversion is initiated from application user interface reaches the Room database and stores the record of file.
Driver/precondition	• Application is running. • Input path is given.
Test steps	1. Upload the valid file and clicks “Convert”. 2. Application calls startConversion (). 3. Stores successful results in database.
Input	inputUri = selected file URI
Expected Results	File is converted successfully and stored in the Room database.
Actual Results	Pass
Remarks	Pass

5.3.2 Test Case 2

Table 61 Integration Test Case 2

Testcase ID	IT 2
Requirement ID	R3
Title	Frontend →Backend →Room Database: Compression
Description	Test the full integration path from uploading the file to using library to

	Room database storage.
Objective	Verify that the compression is initiated from application user interface reaches the Room database and stores the record of file.
Driver/precondition	• Application is running. • Input path is given.
Test steps	1. Upload the valid file and clicks “Compress”. 2. Application calls startCompression (). 3. Stores successful results in database.
Input	inputUri = selected file URI
Expected Results	File is compressed successfully and stored in the Room database.
Actual Results	Pass
Remarks	Pass

5.3.3 Test Case 3

Table 62 Integration Test Case 3

Testcase ID	IT 3
Requirement ID	R4
Title	Frontend →Backend →Room Database: Text Extraction
Description	Test the full integration path from uploading to extract the text file to using library or API to Room database for storing .txt file.

Objective	Verify that extraction initiated from application user interface reaches the Room database and stores the record.
Driver/precondition	• App is running. • Valid input file path is given.
Test steps	1. Upload the valid file and clicks “Extract Text”. 2. Application calls extractTextFromImage (). 3. Stores successful results in database.
Input	imageUri = selected image URI, mimeType = image/jpeg or image/png
Expected Results	Text is extracted successfully and record is stored in Room database.
Actual Results	Pass
Remarks	Pass

5.3.4 Test Case 4

Table 63 Integration Test Case 4

Testcase ID	IT 4
Requirement ID	R5
Title	Frontend →Backend →Room Database: Pdf Tools
Description	Test the full integration path from uploading pdf to using a library for operation to successfully storing pdfs in Room database.
Objective	Verify that both pdf operations start from application user interface and

	stores record in Room database.
Driver/precondition	• Application is running. • Valid pdf file is given.
Test steps	1. Upload one or more pdf files and press “Merge” or “Split”. 2. Application calls the respected function. 3. Stores in Room database.
Input	Merge: selectedUris = [file1.pdf, file2.pdf] Split: selectedUris = [file.pdf]
Expected Results	Pdf is merged or split in Room database.
Actual Results	Pass
Remarks	Pass

5.3.5 Test Case 5

Table 64 Integration Test Case 5

Testcase ID	IT 5
Requirement ID	R6
Title	Feedback Submit→ Firebase Firestore →Developer View
Description	Test the full integration from submission of feedback to storing it in Firebase Firestore.
Objective	Verify that feedback is submitted is submitted from application user

	interface and stored at Firebase Firestore for developer view.
Driver/precondition	• Device has internet connection. • Firebase Firestore is connected.
Test steps	1. Open Settings. 2. Open the Feedback dialog. 3. Enter feedback. 4. Tap Submit. 5. FeedbackDialogHelper creates the feedback data. 6. Data is submitted to the feedback Firestore collection. 7. Verify the success message.
Input	Message=”good application”, star rating=5
Expected Results	Feedback is stored in Firebase Firestore.
Actual Results	Pass
Remarks	Pass

5.4 Performance testing

Performance testing was conducted to assess the speed and responsiveness of Hybrid Format Converter. This testing focused on measuring system performance during processes such as conversion, compression, text extraction, history and feedback submission to ensure acceptable response time. The main purpose was to confirm that the system provides reliable performance and a good user experience during regular use.

Table 65 Performance Testing Metrics

Tools/Methods	Metrics Measured	Result/Observation
PostMan POST for text Extraction	Average response time (10 requests)	2.07 seconds averages
Logcat for conversion	Average response time (10 requests)	0.275 seconds averages
Logcat for feedback	Average response time (10 requests)	1.89 seconds averages
Java desktop for conversion	Average response time (15 requests)	5.51 seconds averages

Observations

- Image conversion completed with 20 seconds on average while heavy conversions like audio, video, documents take more time like 4-6 seconds on average.
- Online text extractions take less time like 1073ms while offline text extraction takes more time on an average.
- Firebase Firestore is online so, it takes 2-4 seconds on average.
- Room Database loads history within 1-2 seconds which shows efficiency of fixed records.

API Testing sampling

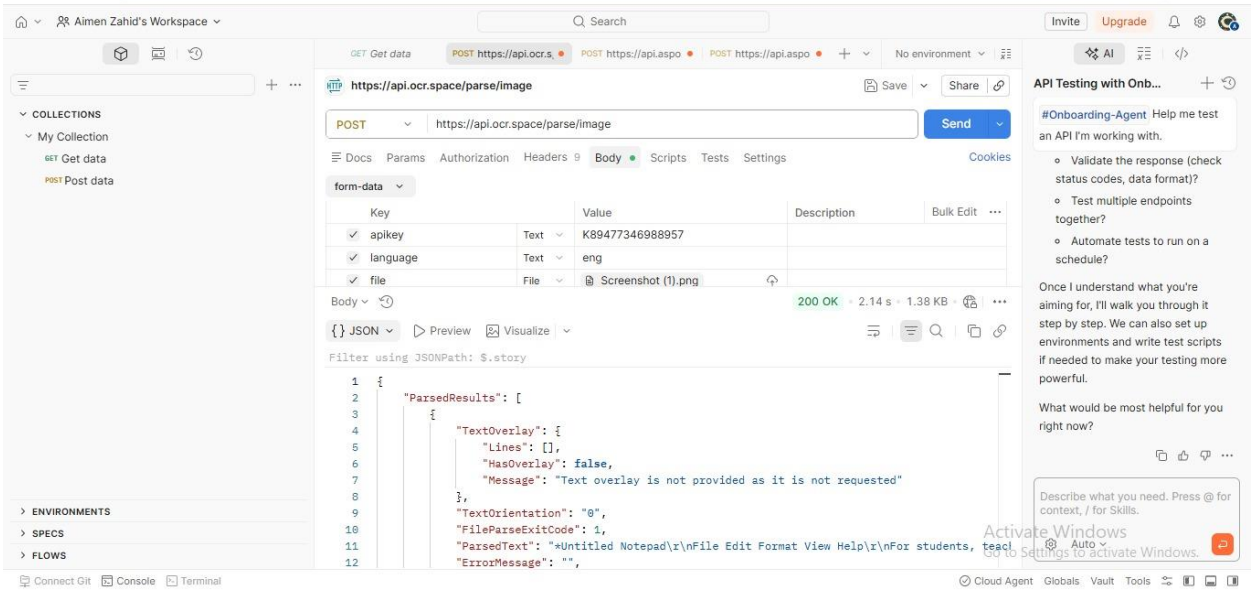


Figure 5-1 Testing Sample 1

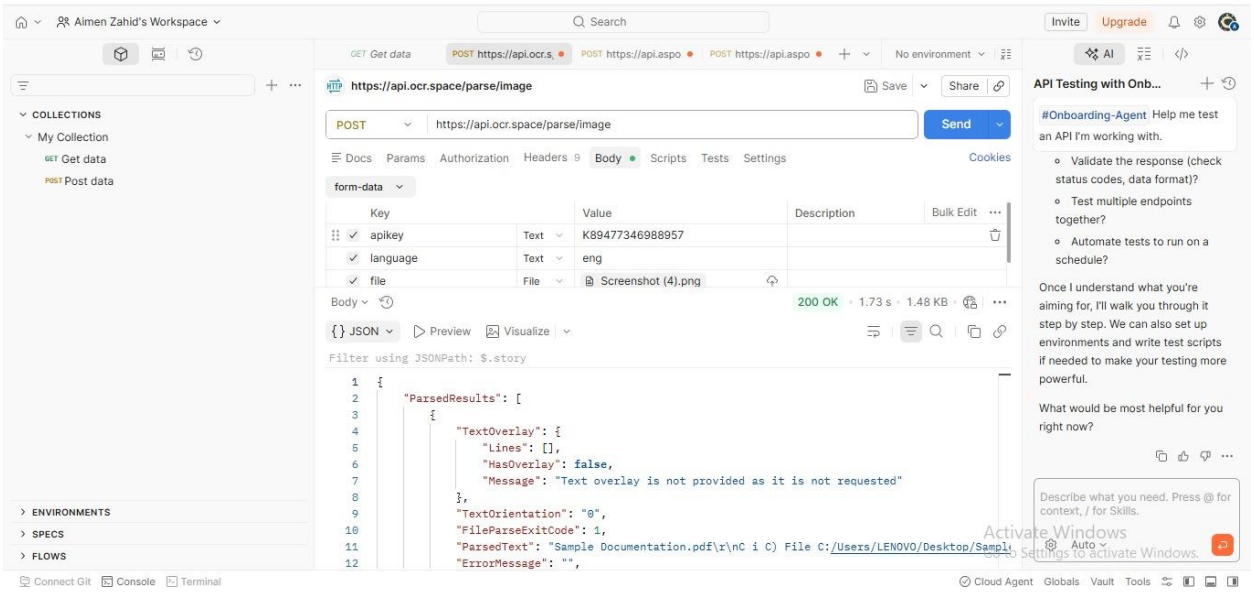


Figure 5-2 Testing Sample 2

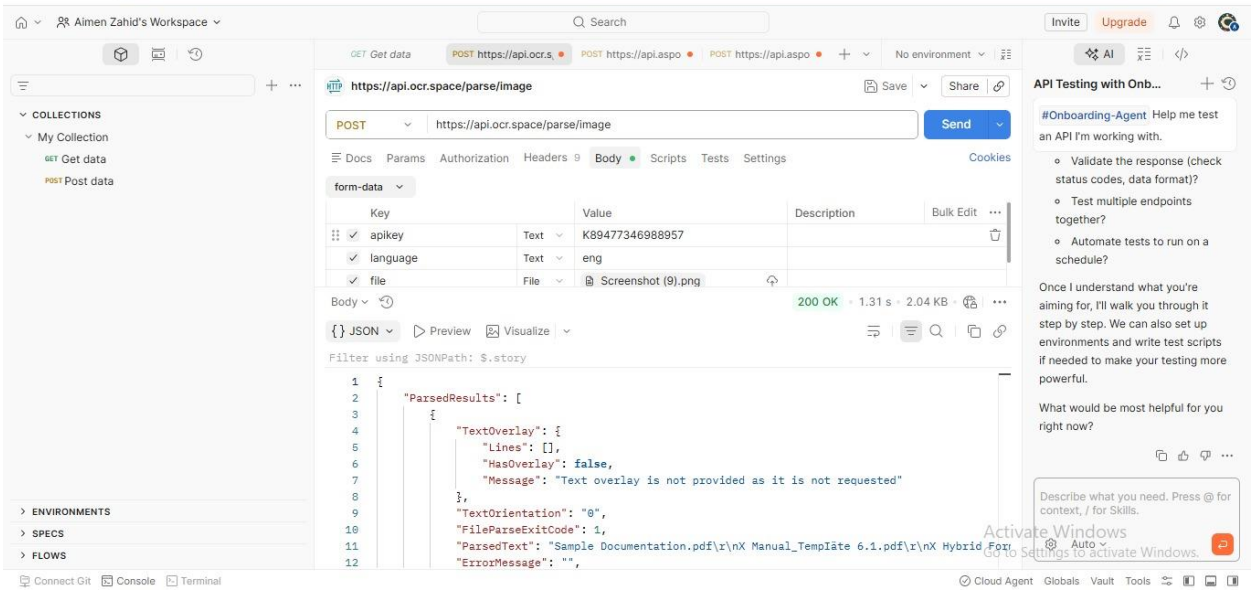


Figure 5-3 Testing Sample 3

5.4.1 Test Case 1

Table 66 Performance Test Case 1

Testcase ID	PT 1
Requirement ID	R2
Title	Conversion-Processing Time
Description	Measures the average time conversion takes for processing and returns the converted file.
Objective	Verify that conversion takes acceptable time.
Driver/precondition	• Application is running. • Internet is required or not based on the advancement of conversion.
Test steps	1. Perform the different types of conversion of image, document, audio and video.

	2. Records the response time in logcat. 3. Calculate the average response time.
Input	inputUri = selected file URI
Expected Results	Acceptable average response time.
Actual Results	Pass
Remarks	Pass

5.4.2 Test Case 2

Table 67 Performance Test Case 2

Testcase ID	PT 2
Requirement ID	R4
Title	Text Extraction-Response time
Description	Measures the average time it takes for online text extraction.
Objective	Verify that online text extraction completes within an acceptable time.
Driver/precondition	Internet connection is needed.
Test steps	1. Perform online text extraction 10 times. 2. Record response time for each. 3. Calculate average response time for text extraction.
Input	imageUri = selected image URI, mimeType = image/jpeg or image/png

Expected Results	Acceptable average response time.
Actual Results	Pass
Remarks	Pass

5.4.3 Test Case 3

Table 68 Performance Test Case 3

Testcase ID	PT 3
Requirement ID	R6
Title	Feedback Submission -Response Time
Description	Measures the average time from tapping submit on feedback to receiving feedback submitted message.
Objective	Verify feedback submission completes within an acceptable time.
Driver/precondition	• Device has internet connection. • Firebase Firestore is connected.
Test steps	1. Navigate to feedback through settings. 2. Submit feedback 10 times. 3. Record time from submit feedback to confirmation message. 4. Calculate the average time.
Input	message=” helpful application”
Expected Results	Acceptable average response time.

Actual Results	Pass
Remarks	Pass

5.4.4 Test Case 4

Table 69 Performance Test Case 4

Testcase ID	PT 4
Requirement ID	R7
Title	History-loading Time
Description	Measures the average time history takes to load from the database.
Objective	Verify that the history takes appropriate time for loading the last 5 files.
Driver/precondition	• Application is running. • Internet is not needed.
Test steps	1. Navigate to history feature. 2. Application fetches last 5 files from Room database to display.
Input	Files already exist in Room database.
Expected Results	Acceptable average response time.
Actual Results	Pass
Remarks	Pass

5.5 Summary

This chapter focuses on testing and evaluation of our Hybrid Format Converter. Different testing methods, including unit testing, functional testing, integration testing, and performance testing, were conducted to verify that each module works correctly both individually and as part of the complete system. These tests were conducted to examine system features, verify module interaction, and measure overall performance.

Chapter 6

System Conversion

6. System Conversion

System conversion is the process of implementing a developed system and making it fully operational for users. In the Hybrid Format Converter, it involves deploying the new system, integrating its features, and ensuring smooth functionality, user adaptation, and reliable performance.

6.1 Conversion method

For the implementation of the Hybrid Format Converter, the selected conversion method is phased conversion.

6.1.1 Phased Conversion

In the phased conversion approach, the Hybrid Format Converter application is deployed gradually, one module at a time. Each module is developed, tested, and made available to users before the next module is introduced. This ensures that every part of the application is stable and functional before expanding the feature. It reduces the risk of system failures.

Justification

Reduced Risk
Since only one module is deployed at a time, so any problem related to that module will be resolved before the development of the next module. In this way, it would not affect the entire application.

Focused Testing
Each module is tested separately like conversion, compression, extraction and pdf tools, without the interference of other modules, allow better testing results for each module.

User feedback loop
From phase 1, user can provide feedback as new module is added. This allows each module to be improved before moving to next phase.

Manageable Development
Deploying our project in phases, help divide work among team members. It helps tracking the progress and meet deadlines.

6.1.2 Phases Distribution

Table 70 Phase Distribution

Phases	Details
Phase 1	It involves the core application module, conversion module which allows the conversion of different formats along with their feature.
Phase 2	It involves the file compression module which reduces the sizes of images, audios, document and videos.
Phase 3	It involves the online text extraction.
Phase 4	It involves the pdf tools such as pdf merge, pdf split and conversion between image and pdf to manipulate the pdfs.
Phase 5	It has feedback and rating module which allows the user to give feedback to the system.

6.2 Deployment

Deployment involves preparing and transferring the Hybrid Format Converter application from the development environment to an operational environment for end users. This includes building the APK, configuring required offline libraries and tools and ensuring that all conversion modules (such as file format conversion, compression, and PDF tools) function correctly. The application is then installed and tested on Android devices to ensure smooth performance and reliability.

6.2.1 Deployment Steps

Table 71 Deployment Steps

Step No	Task	Details
1	Build APK/Bundle	Generate signed APK using Android Studio for production deployment.
2	Core Setup	Integrate all required offline libraries (PDF Box, FFmpegkit and Apache POI) ensure basic app structure is ready.
3	Database Configuration	Set up Room Database for storing files, conversion history, and user data locally.
4	Firebase Setup	Configure Firebase Firestore for feedback submission and ensure backend connectivity.
5	phase 1 Deployment (File Conversion)	This phase deployed the core file conversion functionalities, enabling users to convert files between supported formats accurately.
6	phase 2 Deployment (File Compression)	Deploy file compression functionality and test file size reduction without data loss.
7	phase 3 Deployment (Text Extraction)	This phase deployed the text extraction functionality, enabling users to extract text from selected files.
8	phase 4 Deployment (Pdf Tools)	This phase deployed the PDF tools functionality, enabling users to perform basic PDF operations such as pdf merge and pdf split.
9	phase 5 Deployment (Feedback Module)	Deploy feedback system integrated with Firebase Firestore and verify successful submission.

10	Integration Testing	Test interaction between all modules to ensure smooth workflow
11	Device Compatibility	Test application on multiple Android devices for performance and compatibility.
12	Final Deployment	Release fully integrated and stable version of the application.

6.2.2 Data Conversion

Table 72 Data Conversion

Step No	Task	Details
1	Data Cleaning	Ensure selected files are valid, remove unsupported or corrupted files, and handle the processing issues before processing.
2	Data Transformation	Convert the files into required formats and prepared meta data.
3	Data Validation	Verify that converted and compressed files are accurate, complete and not corrupted before downloading.
4	Data Loading (Local-Storage)	Store converted and compressed files along with history in Room database or device storage.
5	Data Loading (Cloud-Feedback)	Upload user feedback data to Firebase Firestore.

6.2.3 Training

In the Hybrid Format Converter, it involves guiding users on how to access the application, perform file conversions, and utilize features such as compression and PDF tools. The training ensures users can operate the system confidently, leading to smooth adoption and reduced operational errors.

Table 73 Training Instruction 1

Step No	Instructions
1	Open the Hybrid Format Converter application in your android device.
2	On the home screen, select the desired conversion type (images, audios, videos and documents).
3	Choose the format for that conversion.
4	Permission box will appear, grant permission for file access.
5	Upload the file you want to convert and tap “Convert”.
6	Conversion will be performed and be available to download.
7	Successfully processed file will be appears in history.

Table 74 Training Instruction 2

Step No	Instructions
1	Open the Hybrid Format Converter application in your android device.
2	On the home screen, select the text extraction feature.

3	Choose the format for the Extraction.
4	Permission box will appear, grant permission for file access.
5	Upload the file you want to extract the text from and tap “Extract”.
6	Extraction will be performed; user may copy or download the .txt file.
7	Successfully processed file will appear in history.

6.3 Post Development Testing

Post-deployment testing was conducted to validate that the Hybrid Format Converter system was correctly installed, configured, and functioning as intended. The objective was to ensure successful file conversion, module functionality, system stability, and overall user experience after deployment.

6.3.1 Post Deployment Testing Steps

Table 75 Post Deployment Testing Steps

Step No	Text Activity	Expected Outcome
1	App Launch	App opens without crashing. Home screen displays all modules correctly: Conversion, Compression, Text Extraction, PDF Tools, History, Settings.
2	Offline Conversion	Select JPG → PNG (offline format). Upload a JPG file. Verify converted PNG is returned and downloadable without internet.
3	Online Conversion	Select an advanced format (e.g., DOCX → PDF) with internet active. Verify Cloud API responds and converted file is returned.
4	File Compression	Select Image compression. Upload an image. Verify fileSizeAfter < fileSizeBefore and compressed file is

		downloadable.
5	Online Text Extraction	Turn on internet. Upload an image with text. Verify Cloud OCR extracts text correctly.
6	PDF Merge	Upload two PDF files. Tap Merge. Verify merged PDF is generated and downloadable.
7	PDF Split	Upload a multi-page PDF. Select pages. Tap Split. Verify split PDF contains only selected pages.
8	History	Navigate to History. Verify last 5 download files are displayed with correct names and timestamps.
9	Feedback Submission	Open Settings. Submit text feedback and a star rating. Check Firebase Console to confirm documents saved in feedbacks and ratings collections.
10	Firebase Firestore	Open Firebase Console and verify that feedback and rating documents are correctly stored with timestamps.
11	Storage Permission	Uninstall and reinstall app. On first file operation, verify the permission dialog appears correctly.

6.4 Challenges

During deployment and post-deployment phases of the Hybrid Format Converter, several technical and operational challenges were encountered. Below is a list of the main challenges along with their applied solutions.

Table 76 Challenges and Solutions

Challenge	Description	Solution Implemented
Offline Library Integration Issues	Different file conversion libraries (PDF, DOCX, and Image) caused compatibility and dependency conflicts.	Carefully selected lightweight libraries.

File Format Conversion Accuracy	Some formats (e.g., PDF to Word) were not converting with proper formatting.	Tested multiple libraries to improve output accuracy.
Storage Permission Issues	App failed to access files due to Android storage permission restrictions.	Implement permission handling so that app can access the files only when user give the permission
Limited Device Storage	Converted files increased storage usage on low-end devices.	Added file compression option and allowed users to delete files after download.
Unsupported File Formats	Some file types were not supported for conversion.	Ensure feedback mechanism so that, users can easily state their required formats.
Poor internet access.	Affect online conversions and online text extraction.	Ensure maximum conversions and text extraction without internet.

6.5 Summary

This chapter explained the deployment process of the Hybrid Format Converter, including system implementation, user training requirements, and data conversion. It also highlighted the major challenges faced during deployment and development, along with the practical solutions to ensure successful system operation.

Chapter 7

Conclusion

7. Conclusion

This chapter concludes the Hybrid Format Converter Project. It presents the requirements traceability matrix linking system requirements with design and testing provides an overall conclusion, and discusses future improvements.

7.1 Evaluation

Table 77 Progress Evaluation

Objectives	Status
Design and develop a hybrid Android application for files conversions, compression and text extraction.	Completed
Provide conversion formats for images, documents, audios and videos.	Completed
Enable basic conversions in offline mode and advance conversions.	Completed
Implement history feature where end-users can access their last 5 download files.	Completed
Ensure offline conversion so user can perform conversion with no internet access.	Completed
Provide text extraction feature using Optical Character Recognition (OCR) for online text extraction to improve usability.	Completed
Implement compression feature to reduce the size of large files.	Completed
Provide pdf tools such as pdf merging, pdf splitting and conversion between pdf and image.	Completed
Provide a simple and user-friendly user interference.	Completed
To include a feedback mechanism for user to share suggestion or request new features.	Completed

7.2 Traceability Matrix

Table 78 Traceability Matrix

Requirement ID	Requirement Description	Design Specification	Code	Test ID
R1	The system must allow the user to convert files into different formats. (by uploading a file and downloading the processed file after operation)	Component ”Conversion Module”	ConversionActivity.java	UT1, UT2, FT1, IT1
R2	The system must allow the user to reduce the size of their files. (by uploading a file and downloading the processed file after operation)	Component ”Compression Module”	CompressionActivity.java	UT3, FT2, IT2
R3	The system must allow the user to perform text extraction. (by uploading a file and downloading the processed file after operation)	Component ”Text Extraction Module”	Text ExtractionActivity.java	UT4, UT5, FT3, IT3
R4	The system must allow the user to manipulate the pdf by choosing the functionalities such as pdf merge and pdf	Component ”PDF Module”	PdfToolsActivity.java	UT6, UT7, FT4, IT4

	split.			
R5	The system must allow the user to give feedback to the system to increase usability.	Component ”Feedback Module”	FeedbackDialog Helper.java	UT9, FT6
R6	The system must allow the user to access last 5 download files and can delete, rename, share and download.	Component ”History Module”	HistoryActivity.java	UT8, FT5, IT5

7.3 Conclusion

The Hybrid Format Converter successfully achieved its objective by developing a mobile application that provides the multiple file processing functionalities like conversion, compression, text extraction and pdf tools. The system eliminates the need of using separate applications for different functionalities. Our system increases the usability by offering multiple functionalities in one application. It also integrates the Room Database and Firebase Firestore for managing the history and storing user’s feedback.

Table 79 Objectives and System Functionalities

Objective ID	Defined Objective	System Functionality
1	Conversion, Compression and Text Extraction	Provide separate modules to perform conversion, to extract the text, to reduce the size of files.
2	Formats	Provide different formats for images, documents, audios and videos.
3	Hybrid Conversion	Provide basic offline conversion and uses internet for advanced conversions.

4	History	Allow users to access last 5 download files.
5	Hybrid Text Extraction	Uses internet for online text extraction.
6	Compression	Provides a module to reduce the size of files.
7	Pdf Tools	System also provides tools for pdf manipulation like pdf merge and pdf split.
8	Feedback	Allows user to give feedback to the system.

7.4 Future Works

- After the completion of this project, we will focus on iOS version of this application.
- Payment feature will be added for advanced conversion.
- Admin panel will be developed to manage the application.
- Advanced pdf tools will be added like password protection and water marking.

References

Book

- (2015). In I. Sommevillie, *Software Engineering(10th Edition)* (pp. 120-321). London: Pearson Education.

Web Resources

- Husnain, M. (n.d.). *SOW programming*. Retrieved 11 2025, from Youtube: https://www.youtube.com/@SOWProgramming

- Khan, H. A. (n.d.). *Code with Harry*. Retrieved 12 2025, from Youtube: https://youtu.be/mXjZQX3UzOs?si=0WKRtXyYKNjo5B2K

- Developers, A. (n.d.). *Material Design Guidelines*. Retrieved 4 30, 2026, from https://developer.android.com/design

Appendix A: Use Case Description

Use Case 1

Use Case ID	UC-1
Use Case Name	Perform File Conversion
Actors	End User
Description	This use case allows the user to convert the uploaded file in different formats.
Trigger	User clicks “Convert”.
Pre-Conditions	The user must upload the file.
Post Condition	The converted file will be generated and available for user to download.
Normal Flow	1. User opens the app. 2. User selects the conversion. 3. User selects the I/O format. 4. User will upload the file. 5. User selects “Convert”. 6. File is converted and available to download.
Alternative Flows	If conversion failed, user will be shown an error message.
Business Rules	Only supported file will be converted.
Assumptions	• Device has enough storage. • Device permission for file access is granted.

Use Case 2

Use Case ID	UC-2
Use Case Name	Perform File Compression
Actors	End User
Description	This use case allows the user to compress the uploaded file in different formats.
Trigger	User clicks “Compress”.
Pre-Conditions	The user must upload the file.
Post Condition	The compressed file will be generated and available for user to download.
Normal Flow	1. User opens the app. 2. User selects the compression. 3. User selects the I/O format. 4. User will upload the file. 5. User selects “Compress”. 6. File is compressed and available to download.
Alternative Flows	If compression failed, user will be shown an error message.
Business Rules	Only supported file will be compressed.
Assumptions	• Device has enough storage. • Device permission for file access is granted.

Use Case 3

Use Case ID	UC-3
Use Case Name	Text Extraction

Actors	End User
Description	This use case allows the users to extract the text from supported files.
Trigger	User clicks “Text Extract”.
Pre-Conditions	The user must upload a supported file for text extraction.
Post Condition	The.txt file () will be generated and available for user to download.
Normal Flow	1. User opens the app. 2. User selects the Text Extraction. 3. User selects the I/O format. 4. User will upload the file. 5. User selects “Text Extract”. 6. Text is extracted and saved in text file which is available to download.
Alternative Flows	If Extraction failed, user will be shown an error message.
Business Rules	Only supported files will be extracted.
Assumptions	• Device permission for file access is granted. • Internet is available for online Text Extraction if required.

Use Case 4

Use Case ID	UC-4
Use Case Name	Pdf Tools
Actors	End User
Description	This use case allows the end users to perform Pdf operations such as Pdf merge and Pdf split.

Trigger	User selects the "Merge" for merging the Pdf and "Split" for splitting the Pdf.
Pre-Conditions	User must upload the multiple Pdf files for merging and single Pdf file to split.
Post Condition	Single pdf will be generated in the result of pdf merge and multiple pdfs will be generated in the outcome of pdf split.
Normal Flow	1. User opens the app. 2. User selects the Pdf Tools. 3. User selects desired tool. 4. User will upload single or multiple Pdfs for different operation. 5. User selects the "Merge" for merging the Pdf and "Split" for splitting the Pdf. 6. Merged pdf or split pdfs will be available for user to download.
Alternative Flows	If operation failed, user will be shown an error message.
Business Rules	Only Pdf files are allowed.
Assumptions	• Device permission for file access is granted. • User must have more than one pdf for merging.

Use Case 5

Use Case ID	UC-5
Use Case Name	Submit Feedback
Actors	End User
Description	This use case allows the user to give feedback or ratings to the system.
Trigger	User selects "Submit Feedback".

Pre-Conditions	Application must be installed and running.
Post Condition	User's feedback will be stored on Firebase Firestore.
Normal Flow	1. User opens the application. 2. User navigated to the home screen. 3. User selects the "settings". 4. User selects written feedback or ratings. 5. For written feedback, user will be shown a text box to fill while for ratings, user will be shown 5 stars to fill. 6. User fills the text box or ratings. 7. User selects "Submit Feedback"
Alternative Flows	If internet is not available user will be shown an error message.
Business Rules	Text box for written feedback and 5 stars cannot be empty.
Assumptions	Internet is available to store the feedback in Firebase Firestore.

Use Case 6

Use Case ID	UC-6
Use Case Name	View History
Actors	End User
Description	This use case allows the user to access previously processed files including conversion, compression, text extraction and pdf tools.
Trigger	User selects the "History" from navigation menu.
Pre-Conditions	The user must have previously performed at-least one file operation.
Post Condition	The user can view and access their file processing history records.

Normal Flow	1. User opens the application. 2. User navigated to the home screen. 3. User selects the “History” option. 4. System retrieves the previously processed file records. 5. User can download, rename, delete and share history records.
Alternative Flows	If no previous record exists, system displays an empty history message.
Business Rules	History must store file operation records. User may mange history records through available action.
Assumptions	• The system has successfully stored previous operations. • User data is maintained.

Use Case 7

Use Case ID	UC-7
Use Case Name	File Select
Actors	End User
Description	This use case allows the user to upload the file from device to perform desired operation.
Trigger	User clicks “Select File”.
Pre-Conditions	File must be available for user to upload.
Post Condition	Selected operation will be performed on uploaded file.
Normal Flow	1. User opens the application. 2. User selects the feature. 3. User selects the desired format. 4. User clicks “Upload File”.

	5. User chooses the file from the device. 6. File is uploaded.
Alternative Flows	If user does not grant the permission to access the file, user will be redirected to the previous screen.
Business Rules	App must have storage permission to access the file.
Assumptions	File must be available to upload.

Use Case 8

Use Case ID	UC-8
Use Case Name	Download File
Actors	End User
Description	This use case allows the user to download the processed files such as compression, conversion, text extraction and pdf files.
Trigger	User selects the “Download” option after the completion of file operation.
Pre-Conditions	The requested file must be successfully processed and available for download.
Post Condition	The file is successfully downloaded and saved to device storage.
Normal Flow	1. User completes an operation. 2. User selects the “Download” option. 3. System downloads and saves the file to device storage. 4. User receives confirmation of successful download.
Alternative Flows	If an error is occurred during download, user is notified.
Business Rules	• Only successful files can be downloaded.

	• Device permission is required.
Assumptions	• The user has sufficient device storage. • Device permissions are granted.

Use Case 9

Use Case ID	UC-9
Use Case Name	Share File
Actors	End User
Description	This use case allows the user to share the processed files from history.
Trigger	User clicks “Share” option for a processed file in history.
Pre-Conditions	The selected file must exist and be available for sharing.
Post Condition	File is successfully shared.
Normal Flow	1. User opens the application. 2. User access history items. 3. History displays last 5 successfully processed files. 4. User selects the file, user wants to share. 5. User clicks “Share” option. 6. System opens the available platform. 7. User selects the preferred sharing method. 8. System attaches the file. 9. User confirms sharing. 10. File is shared successfully.
Alternative Flows	If file is unavailable, sharing is cancelled.

Business Rules	• Only downloaded files can be shared. • User permissions are required for sharing access.
Assumptions	• File exists and is accessible. • Device permissions are granted.

Use Case 10

Use Case ID	UC-10
Use Case Name	Delete File
Actors	End User
Description	This use case allows the user to delete the processed files from history.
Trigger	User clicks “Delete” option for a processed file in history.
Pre-Conditions	File must exist in the application.
Post Condition	File is removed from the database.
Normal Flow	1. User opens the application. 2. User access history items. 3. History displays last 5 successfully processed files. 4. User selects the file, user wants to delete. 5. User clicks “Delete” option. 6. File is deleted successfully from the database.
Alternative Flows	If user clicks the cancel, file is not deleted.
Business Rules	• File is removed permanently. • Confirmation is required before deletion.
Assumptions	• File exists and is accessible.

	• Device permissions are granted.

Use Case 11

Use Case ID	UC-11
Use Case Name	Rename File
Actors	End User
Description	This use case allows the user to rename an existing file.
Trigger	User clicks “Rename” option.
Pre-Conditions	The file must be available in the application.
Post Condition	The file name is successfully updated in the database. The updated file name is reflected in the history.
Normal Flow	1. User opens the application. 2. User access history items. 3. History displays last 5 successfully processed files. 4. User selects the file; user wants to rename. 5. User selects “Rename” option and a text box is shown. 6. User writes the new name in the text box. 7. User clicks the “Rename” button. 8. File successfully renamed.
Alternative Flows	If user clicks the cancel, file is not renamed.
Business Rules	File name must not be empty.
Assumptions	• File exists and is accessible to rename. • Device permissions are granted.

Appendix B: Coding Standards

1. Variable Naming Conventions

- Consistent naming is followed throughout the application.
- The following conventions are used based on the context:
  Snake-case: Used for database fields and configurations.
  Pascal Case: Used for class names (e.g., ConversionActivity, CompressionActivity, FeedbackDialogHelper).
  Camel Case: Used for variables and function names (e.g., doMerge(), doSplit(), startConversion ()).
  Hungarian Notation: Used occasionally for clarity.

2. Visual Scope Indicators

- Variables and constants are named with clear scope indicators:
  selectedFiles
  filePath
  fileName

3. Read-Only and Immutable Fields

- Final keyword in Java is used for immutable values.
- File-related attributes (e.g., file name, format, size) are protected from modification after initialization.
- Read-only properties are exposed through getter methods only, without public setters.

4. Function Naming and Behavior

- All function names clearly describe their behavior:
  startConversion()
  startCompression()
  extractTextfromImage()

doSplit()

loadHistory()

domerge()

5. Commenting Style

- Block and inline comments are included only where logic is not immediately self-explanatory.
- Example

  /* This function performs file conversion */

  void startConversion() {

  // conversion logic here

  }

6. Indentation and Code Structure

- Proper indentation is used to clearly define:
- Control Structures (if-else, loops)
- Function boundaries
- Class structure
- This improves readability and maintainability.

7. File and Folder Naming

- Classes and their corresponding files are named identically:

  ConversionActivity.java

  CompressionActivity.java

  TextExtractionActivity.java
- Package structure separates application layers:

  layout/, drawable/, themes/, colors/, java/

8. Class Structure and Separation of Concerns

- Each class has a single responsibility.
- The application follows **MVVM architecture**

  View → UI screens

  ViewModel → handles UI logic

  Repository → manages data operations

  Model → represents data structure
- Data Structure

  Room Database (history and offline data)

  Firebase Firestore (feedback storage)

9. Access Modifiers and Scoping

- Fields and methods are declared with minimum required visibility (private, public).
- Sensitive operations (file handling, database access) are restricted to appropriate layers.

10. Cohesion and coupling

- High cohesion is ensured by grouping related functionalities within modules:

  Conversion Module

  Compression Module

  Text Extraction Module
- Low coupling is achieved by:

  Separating modules logically

  Using repository pattern for data handling

  Isolating API and database interactions

11. DRY Principle

- Code reuse is prioritized.

- Common functionalities such as:

  selectFile()

  validateFile()

  are reused across modules.
- Utility/helper classes are used for shared logic.
- Redundant code is avoided through modular design.

12. Single Responsibility Principle

- Each

  Method performs a single task (e.g., conversion, compression).

  Class handles one responsibility (e.g., Feedback, History).

  Module represents one feature of the system.

13. Extensibility and Maintainability

- Open/Closed Principle is followed
- Code is open for extension but closed for modification.
- New features (e.g., additional file formats, new PDF tools) can be added easily.
- Reusable components and modular structure support long-term maintenance.

14. Exception Handling and Logging

- All exceptions are handled using try-catch blocks.
- Errors such as:

  Invalid file format

  Network failure during online conversion

  File processing errors are properly managed.

  Logging is done using Android logging utilities with meaningful messages.
- **Example:**

  Log.e("DocumentConverter", "Conversion failed: " + e.getMessage());

Appendix C: Application Prototype

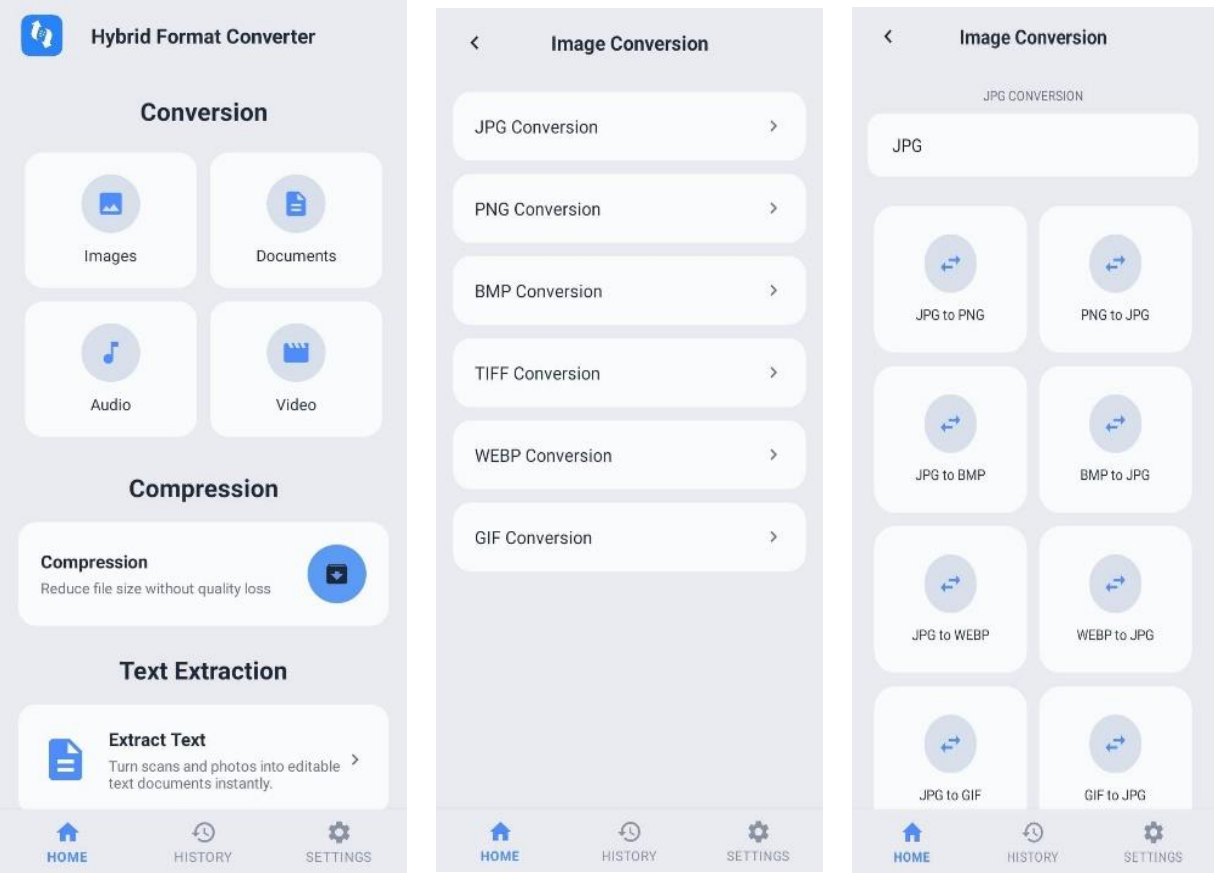
Application Overview

The application prototype provides a visual representation of the Hybrid Format Converter system's layout, functionality, and user interaction flow. It covers the following core modules:

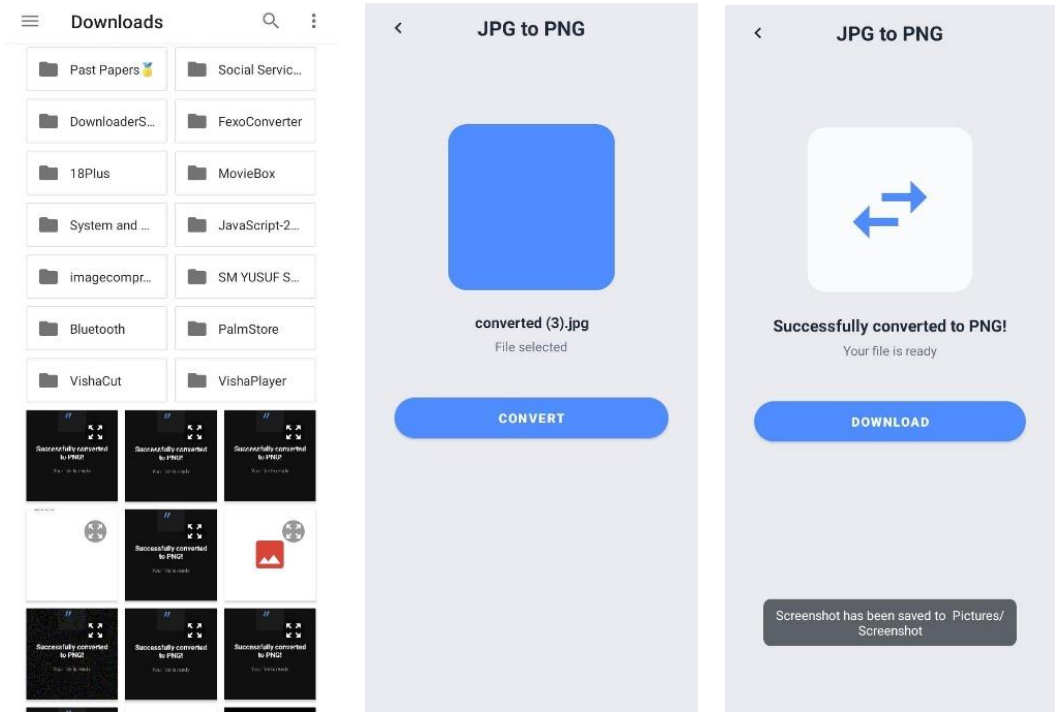
- Conversion Module
- Compression Module
- Text Extraction Module
- Pdf Tools Module
- History Module
- Settings Module

Screenshots of key Interfaces

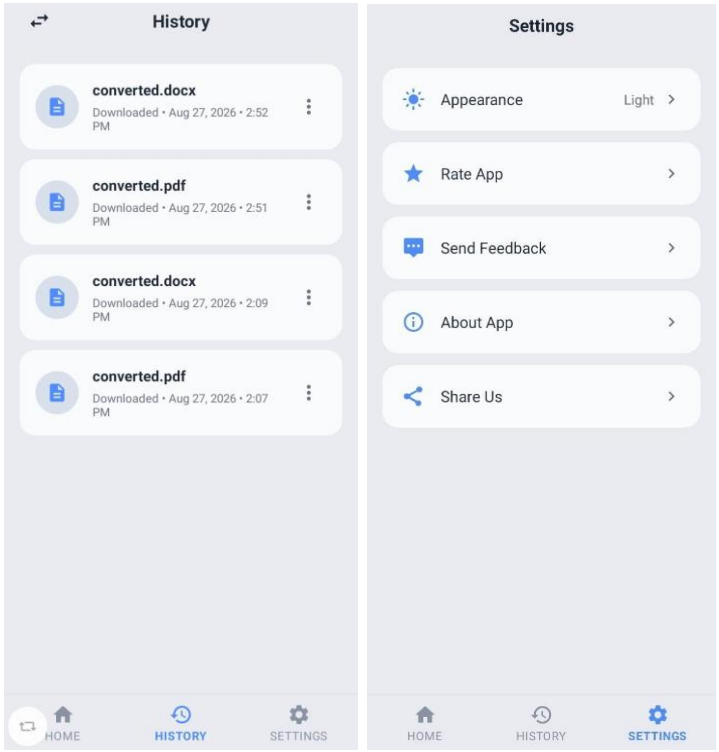
Home Screen and Format Screen



File Upload, permission Screen and successful conversion screen



History and settings Screen



Appendix D: Application's provided formats

Conversion

Images	
JPG↔PNG	Offline
JPG↔BMP	Offline
JPG↔WEBP	Offline
JPG↔GIF	Offline
JPG↔TIFF	Offline
JPG↔PDF	Offline
PNG↔WEBP	Offline
PNG↔BMP	Offline
PNG↔GIFF	Offline
PNG↔TIFF	Offline
PNG↔PDF	Offline
WEBP↔GIF	Offline
GIF↔WEBP	Offline
Audio	
MP3 ↔WAV	Offline
MP3↔AAC	Offline
MP3↔M4A	Offline
MP3↔OGG	Offline
MP3↔OPUS	Offline
MP3↔FLAC	Offline
MP3↔WMA	Offline
Video	
MP4↔AVI	Offline
MP4↔MOV	Offline
MP4↔FLV	Offline
Documents	

DOCX↔PDF	Online
DOCX↔HTML	Offline
DOCX↔TXT	Offline
PDF↔TXT	Offline
PDF→JPG	Offline
PDF→PNG	Offline
MD→PDF	Offline
HTML→PDF	Offline
EPUB→TXT	Offline
HTML↔MD	Offline
EPUB→HTML	Offline
XLSX↔CSV	Offline
XLSX↔JSON	Offline
XLSX↔XML	Offline
CSV↔JSON	Offline
CSV↔XML	Offline

Compression

Images Compression	
JPEG/JPG	Offline
PNG	Offline
WEBP	Offline
GIF	Offline
Audio Compression	
MP3	Offline
WAV	Offline
AAC/M4A	Offline
OGG/OPUS	Offline
FLAC	Offline
Video Compression	

MP4	Offline
MOV	Offline
AVI	Offline
FLV	Offline
Documents Compression	
PDF	Offline
DOCX	Offline
XLSX	Offline
PPT	Offline
TXT	Offline
CSV	Offline

Text Extraction

Images	
JPG	Online
PNG	Online
WEBP	Online
BMP	Online

PDF Tools

PDF Merge	Offline
PDF Split	Offline
PDF↔Image	Offline